



Project Report

On

Secure Multi-Tenant Notes App with AI Threat Detection

**Submitted to D Y Patil International University, Akurdi, Pune
in partial fulfilment of full-time degree**

Bachelor of Technology

Submitted By:

Ms. Payal Gaikwad	PRN: 20220802064
Mr. Sanket Bankar	PRN: 20220802151
Mr. Siddhant Mengade	PRN: 20220802365

Under the Guidance of

Mrs. Monika Kapgate

Assistant Professor

School of Computer Science, Engineering and Applications

D Y Patil International University, Akurdi, Pune, INDIA - 411044

[Session 2025-26]



CERTIFICATE

This is to certify that **Ms. Payal Gaikwad** (20220802064), **Mr. Sanket Bankar** (20220802151), **Mr. Siddhant Mengade** (20220802365) are bonafide students of the School of Computer Science Engineering and Applications (SoCSEA), have satisfactorily completed the project work entitled **Secure Multi-Tenant Notes App with AI Threat Detection** submitted to D Y Patil International University, Pune in partial fulfillment for the award of Bachelor of Technology- CSE Cyber Security in the academic year 2025-26. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated into the report deposited in SoCSEA. The project report has been approved as it satisfies the academic requirements in respect of project report prescribed for the said degree.

Mrs. Monika Kapgate
Project Guide

Dr. Sanjay Mohite
Project Coordinator

Prof. (Dr.) Rahul Sharma

Director

School of Computer Science Engineering & Applications

D Y Patil International University, Akurdi

Pune, Maharashtra, India- 411044

Deceleration

We hereby declare that the project entitled **Secure Multi-Tenant Notes App with AI Threat Detection** submitted by us is original and the research work has been carried out by us independently at School of Computer Science Engineering and Applications, under the guidance of Mrs. Monika Kapgate This report has been submitted in the partial fulfillment for the award of degree of Bachlore of Technology. We also declare that the matter embodied in this report has not been submitted by us for the award of any other degree of any other University or Institute.

Name: Payal Gaikwad

PRN: 20220802064

Name: Sanket Bankar

PRN: 20220802151

Name: Siddhant Mengade

PRN: 20220802365

Acknowledgment

We extend our deep sense of gratitude to our respected guide Mrs. Monika Kapgate , for her valuable help and guidance. We are thankful for the encouragement that she has given us in completing this project successfully.

It is imperative for us to mention the fact that the report of project could not have been accomplished without the periodic suggestions and advice of our project guide Mrs. Monika Kapgate.

We are also grateful Dr. Sanjay Mohite, Project Coordinators and to Prof. (Dr.) Rahul Sharma, Director, SoCSEA,, for their valuable contributions and guidance throughout the course of this project.

We are also thankful to all the other faculties for their kind cooperation and help.

With due respect, we express our profound gratitude to our Hon'ble Vice Chancellor, DYPIU, Akurdi, Prof.(Dr.)Manish Bhalla, for his visionary leadership and unwavering support, which have been instrumental in the successful completion of this project. We are truly honored to have had access to the exemplary facilities and resources of the institution under his esteemed guidance.

Last but certainly not the least; we would like to express our deep appreciation towards our family members and batch mates for providing support and encouragement.

Name: Payal Gaikwad

PRN: 20220802064

Name: Sanket Bankar

PRN: 20220802151

Name: Siddhant Mengade

PRN: 20220802365

Abstract

The technology is vital to contemporary information infrastructure due to its ability for scaling up resources remotely at a reduced expense as well flexibility over available data sources. Companies, schools, organizations as well people are beginning to depend more upon web based programs that can be used for keeping confidential data. Nevertheless with growing cloud usage there is a sharp rise of security issues like unauthorised access, data leaks for tenants, vulnerable api's weak authentication and cyber threats. Most traditional cloud storage solutions do not offer adequate user separation, timely supervision as well as smart defense systems against threats.

“ **Secure multi-tenant notes app with ai threat detection**” is an assignment that addresses this problem through building and deploying a safe, scalable server-less cloud app to safeguard customer's information from cyber threats whilst also keeping track for any malicious activity. This solution will be built with current cloud native technology as well as security techniques for saving notes securely; it also includes logins of users, securing apis from malicious attacks on a per-tenant basis along with live tracking by cloud systems.

The system uses several Amazon Web Services (AWS) cloud services such as Amazon Cognito, API Gateway, AWS Lambda, DynamoDB, CloudWatch, and SNS. Cognito from Amazon provides security as well as managing users' identities securely. The API gateway serves to provide an encrypted interface for connecting applications on both front-end, back end clouds. Serverlessness is provided by AWS lambda for dynamic execution of code on an as-needed basis with no need for servers. We use dynamodb, a non-relational (nosql) db to store secure tenant notes. Amazon cloudwatch offers real-time performance measurement along with logs of program executions as well as detection alerts on breaches through amazon simple notification service (sns).

This is based on multi-tenant cloud architecture, wherein every customer would be considered one of many tenants to this platform. Each record saved on this platform corresponds to an individual's id, which guarantees high levels of tenancy segregation. It blocks access of a user to other users' documents; it helps preserve privacy as well secure information stored on clouds. The tenancy-based segmentation is an integral part of this system's key safety features.

One more important aspect that this initiative has introduced are artificial intelligence-based approaches to detecting threats as well as tracking them out effectively. Our platform tracks all API calls to our services as well as user logins, failed login attempts. The suspicious activity is recorded and tracked with cloud watch service. Though this system is based on rules rather than ai techniques to detect threats now, it provides an initial structure that will be used by

more sophisticated intelligence tools later. Monitoring tier offers live insight to app activity as well as threat alerts so that it staff can detect anomalous patterns of operation which may be indicative of a problem.

The app also provides safe CRUD (Create, Read, Update, Delete) functions for managing notes. Authorized persons are able to safely compose, modify, obtain information about, as well as eliminate entries via secure api calls. The JWT token verification process guarantees access to back-end applications is limited exclusively for permitted individuals. Security dashboard is front-end platform for centralised security monitoring which shows encryption status, alerts and unauthorized attempts count as well cloud monitor details.

the test of time and many conditions, such as. Authorized user access, Unauthorized login attempts, Invalid token requests, Cross-tenant access testing, Runtime execution monitoring, Cloud API validation, DynamoDB activity monitoring, SNS alert verification.

Testing proved successful security features of an app to block intrusion attempts while preserving privacy between tenants as well providing reliable data center visibility services. The cloudwatch logs and lambda monitoring graphs were a good indication of successful, fast-running server-less code that had few failures during run-time. An innovation that we are bringing to you through our product development team, integrating cloud security and artificial intelligence based monitoring systems for an easy-to-use learning platform. This is a demonstration of integrating current serverless architecture, live tracking mechanisms as well as security protocols for developing safe cloud native apps. Shadow identification tracking as well as implementation for a honey pot would enhance novelty possibilities within this project more.

The applied skills of. Cloud computing, Cybersecurity implementation, Serverless architecture, Multi-tenant systems, AWS deployment, API security, Real-time monitoring, Threat detection mechanisms.

It is highly relevant to e-learning software, business clouds services for enterprises as well as private information storage apps. Conclusively, this project has proved to be a successful application for implementing cloud security architecture (csa) by employing multi-tenancy features that allow each client's information storage without any interference from other clients' datasets. This research will provide an excellent base to develop more sophisticated, smart-cloud-based cybersecurity solutions that can deal with complex digital attacks as well as meet business needs related to secure data storage on clouds platforms.

Table of Contents

DECLARATION	i
ACKNOWLEDGEMENT	ii
ABSTRACT	iii
LIST OF FIGURES	vi
LIST OF TABLES	vii
1 Introduction	1
1.1 Problem Statement	2
1.2 Objectives	3
1.3 Purpose	4
1.4 Scope	6
1.5 Applicability	8
2 Literature Survey	9
2.1 Literature Survey	9
2.2 Gap Analysis	16
3 Proposed Methodology	17
3.1 System Architecture / Block Diagram	24
3.2 Proposed Methodology	27
3.3 Implementation(Development and Deployment procedures)	33
3.3.1 Flow Diagrams	36
4 Result and Discussion	37
4.1 Implementation Approaches	38
4.2 Pseudo Code	43
4.3 Analysis (graphs/chart)	56
5 Conclusion And Future Scope	62
REFERENCES	67

List of Figures

1.1	Problem Statement	2
1.2	Purpose	5
3.1	System Architecture	24
3.2	System Architecture	25
3.3	System Architecture	26
3.4	Methodology	32
3.5	Flow Diagram	36
3.6	Flow Diagram	36
4.1	Implementation Approaches	39
4.2	Overall implementation strategy	40
4.3	Environment Setup and Technology Stack	41
4.4	Multi-Tenant Data Isolation Implementation	41
4.5	Unauthorized Access Detection	42
4.6	Pseudo Code for User Authentication	43
4.7	Pseudo Code for Create Secure Note	44
4.8	Pseudo Code for Fetch Note	44
4.9	Pseudo Code for Update Notes	45
4.10	Pseudo Code for Delete Notes	46
4.11	Pseudo Code for Unauthorized Attempt Counter	46
4.12	Pseudo Code for AI Threat Detection Logic	47
4.13	Pseudo Code for Shadow Identity Detection	47
4.14	Pseudo Code for SNS Alert System	48
4.15	Pseudo Code for CloudWatch Monitoring	48
4.16	Functional Testing	50
4.17	Secure Note Creation Testing	51
4.18	Fetch Notes Testing	52
4.19	Testing Check	53
4.20	JWT Token Validation Testing	54
4.21	SNS Alert Security Testing	55
4.22	API Response Time Graph	58
4.23	Cloud Resource Usage Graph	59
4.24	Security Event Distribution Pie Chart	60
4.25	Lambda Invocation Analysis	60
4.26	Database Activity Analysis	61

List of Tables

1 Introduction

The vast benefits of cloud computing, such as its greater scalability, flexibility, and reduction in running costs, have made it a go-to solution for today's software applications. Cloud services are the reason many organizations and users store their most sensitive data on cloud platforms as it offers easy access, faster deployment and centralized data management. Meanwhile, the rise of the cloud has also raised security concerns like unauthorized access, insider threats, identity abuse, API abuse and data leakage.

The traditional notes management systems are mostly designed to perform basic actions such as storing and fetching notes. Typically lack of advanced security features like real-time monitoring, audit logging, suspicious activity monitoring or tenant-level data isolation. This can pose potential security risks and unauthorized use concerns for users and organizations.

The challenges are addressed in the proposed project, "Secure Multi-Tenant Notes App with AI Threat Detection," which proposes a secure cloud-based platform for managing notes with AWS. This app uses server-less architecture and runs on Amazon Web Services (AWS). Lambda, API Gateway, DynamoDB, CloudWatch, SNS, IAM. Users are stored by this software. Keep it secure also constantly check for any unusual activity as well as unauthorized accesses. It includes threats tracking, as well as surveillance of a person's double life. Inspired from AI, this system is created for detecting abnormal behavior of users. Unauthorized login and audit trails, as well as live reports of unauthorized access attempts can be accessed to monitor such incidents. CloudWatch Monitors are used to be centrally monitored, and also if there's a security alert then it will send out notifications via SNS. Unusual activity is detected. The scope of this work includes cloud computing, cyber security server-less architecture. Security with AI - all-in-one scalable platform. How it is done best. Secure advanced apps on a cloud, but not managing it yourself would be better than the infrastructure.

1.1 Problem Statement

Another major concern relates to lack of adequate auditing controls and monitors for use here too, please. Many software products do not track their login attempts, whether successful to unsuccessful ones nor does it alert on unauthorized access attempt by an user. It becomes almost impossible both for identifying and analyzing cyber-attacks. This system shall enable users' note storage as well provide proper isolation for its tenancy feature on an edge platform utilizing cloud native architecture principles. The challenges are addressed by proposing AWS serverless services and AI-based threat detection mechanisms to enhance the security and monitoring of cloud-based applications.

Also, conventional notes solutions mainly concentrate upon record keeping of users' information with no support for sophisticated measures like tenancy segregation, logbook registration (audit), identification surveillance along intelligence-based attack detection. It can lead to security breach via exposure information, id frauds as well a threat through fake identities. The security of cloud native application is ensured through regular maintenance, authentication as well allowing individual tenants accessing data within his reach alone effectively.

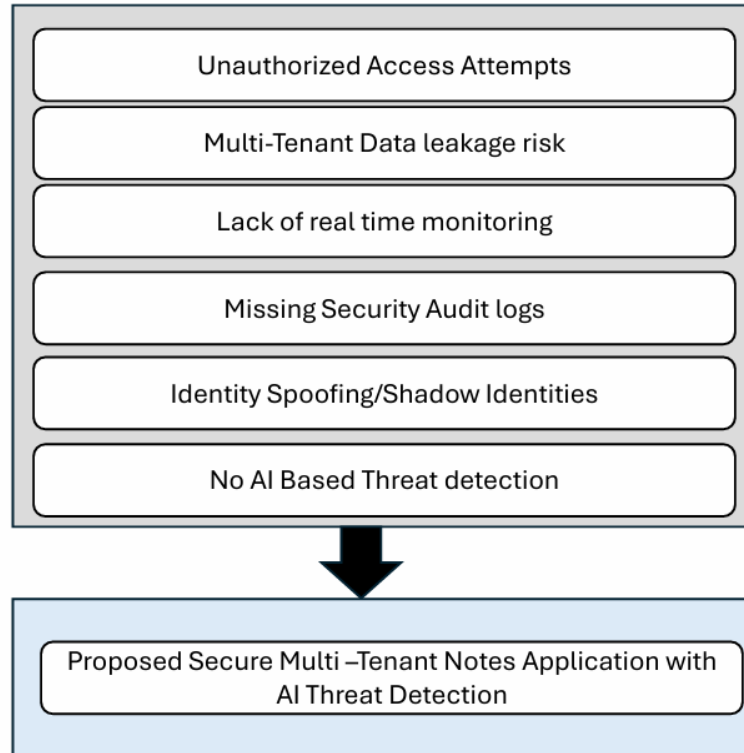


Figure 1.1: Problem Statement

1.2 Objectives

The primary purpose of this project is to develop a secure multi-tenant cloud-based application for note taking with AI-based threat detection capabilities and real-time security monitoring capabilities.

This project aims to create a secure note management application using cloud services provided by AWS. This is done in order to implement a multi-tenant model in which the users can securely access only their own notes. The application supports secure CRUD functions including creating, updating, deleting, and viewing notes.

One of the other key objectives of this project is to deploy this application using a cloud-based serverless architecture that is scalable and easy to manage. AWS Lambda and DynamoDB are used for serverless computing in this cloud-based deployment scenario.

Another objective of the project is to leverage CloudWatch for effective monitoring and log management purposes. SNS notification will be leveraged to trigger immediate security alarms when any suspicious event is observed in the system. This monitoring will happen via auditing the access attempts through logs and metrics.

Moreover, a project based on AI and threat detection concepts as well as shadow identity tracking has been proposed. Dashboards for real-time monitoring will be incorporated to enhance security and monitor activities on the network.

Lastly, the project assists in gaining practical insights into cloud computing and cybersecurity techniques and services provided by Amazon Web Services and serverless deployment techniques.

1.3 Purpose

The main purpose of this project will be to develop a cloud-native application that incorporates the implementation of cybersecurity in the context of serverless computing. In addition, the purpose of the project includes gaining practical experience in cloud security, event processing, and real-time threat monitoring.

Conventional notepad applications generally consider functionality; however, the purpose of this particular project is to go beyond CRUD functionalities to implement various security measures, including:

- Tracking of unauthorized attempts
- Security event monitoring
- Threat analysis using AI techniques
- Audit logging
- Shadow identity detection

The project aims to also show that it is possible to make use of the AWS service integration capabilities to create secure and efficient cloud-based systems. This project looks into the practical application of:

- AWS Lambda
- API Gateway
- DynamoDB
- CloudWatch
- SNS
- IAM

Another important purpose of this project is educational learning. The implementation helps in understanding:

- Cloud deployment models
- Multi-tenant system design
- Threat monitoring systems
- Serverless computing
- Security analytics

The other way this project meets its academic criteria is through innovation, which has been achieved by including AI-assisted security monitoring and shadow identity tracking technology.

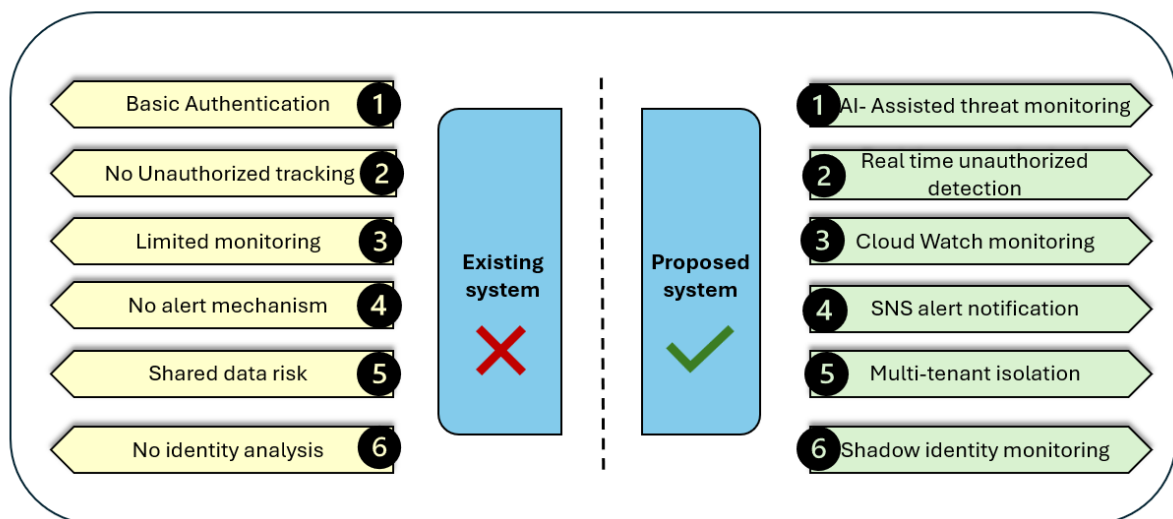


Figure 1.2: Purpose

1.4 Scope

The scope of this project involves the design, development, deployment, and monitoring of a secure multi-tenancy notes management system utilizing AWS cloud services.

The project mainly focuses on:

- Secure user authentication
- Multi-tenant note storage
- Cloud-native deployment
- Real-time monitoring
- Threat detection
- Security event logging

The application allows authorized users to:

- Create notes
- Update notes
- Delete notes
- Retrieve notes securely

The project also includes implementation of:

- Unauthorised access detection
- Security audit logs
- CloudWatch monitoring
- Alert notifications through SNS
- Threat metrics in real-time

This emphasis would continue on the topic of serverless cloud computing in addition to lightweight artificial intelligence-enabled monitoring systems. Training techniques that utilize machine learning as well as advanced artificial intelligence models would fall out of scope for this project.

The project is developed using:

- HTML
- CSS
- JavaScript
- AWS Services

AWS services used:

- AWS Lambda
- API Gateway
- DynamoDB
- CloudWatch
- SNS
- IAM

The project demonstrates practical implementation of cloud security concepts in a real-time environment.

1.5 Applicability

This proposed system can be applied in various practical situations where cloud storage and security monitoring is crucial.

One of the key areas where this system can be employed is note management systems for enterprises. The company can use the system to store all the notes and other confidential information while simultaneously monitoring any attempts to bypass security measures by unauthorized users.

Another area of application of this system will be Software-as-a-Service platforms. This is because the current system adopts the multi-tenant approach of cloud computing which makes it suitable for Software-as-a-Service platforms.

Educational organizations, such as colleges and universities, may also leverage the same systems to manage students' data, educational processes, and security-oriented learning projects. In addition, the project itself will be useful for comprehending cloud security issues and launching a serverless application in practice.

Another application area that should be considered is the creation of cybersecurity monitoring systems. Through the project implementation, one can get an understanding of various security-related elements including threat detection, audit logging, real-time monitoring, and security analysis tools.

In addition, the presented solution may be used to develop any other cloud native secured application, such as document management systems, business dashboards, research-oriented data platforms, or enterprise solutions.

Moreover, the AI-driven approaches for monitoring and detecting shadow identities used in this project can further be developed into AI-based cybersecurity systems capable of detecting abnormal user behavior and potential threats to security.

The application can also be applied in event-driven security situations, where real-time monitoring and threat visualization are needed in cloud infrastructures.

Generally, the project gives an insight into how cloud security can be implemented, as well as cloud serverless computing and real-time monitoring within cloud computing systems.

2 Literature Survey

Overview

Modern applications have been significantly transformed by the use of cloud computing, thanks to its scalability, flexibility, and cost-effectiveness. Modern applications depend largely on cloud computing because of the capability of the technology to ensure fast deployment and resource management. However, the increasing popularity of cloud computing services has created several issues in cyber security such as unauthorized access, spoofing, insider threats, data breaches, and misconfiguration of clouds. These security issues have become prevalent especially in multi-tenant application environments where several users depend on the same cloud computing resources.

The proposed project "Multi-Tenant Notes Application with Cloud Server Architecture and AI Intrusion Detection" is an effort to solve all these problems by using AWS cloud infrastructure and the serverless architecture approach. The application is focused on the issues like security of notes, separation of tenants, logging, monitoring, and AI-based intrusion detection. Literature Review looks into various articles, cloud security solutions, intrusion detection with AI, and multi-tenant security solutions.

2.1 Literature Survey

The following are the Papers

1. AI-Based Intrusion Detection in Cloud Computing Environments

Methodology

The research paper will revolve around the development of artificial intelligence intrusion detection system for cloud computing systems. The intrusion detection system is based on the concept of machine learning to detect any abnormality in terms of traffic and unauthorized access attempts by the user.

Advantages

- Detects suspicious activities in real time
- Improves overall cloud security monitoring
- Reduces the need for manual threat analysis
- Supports automatic alert generation and monitoring

Limitations

- Requires large datasets for training the model
- Increases computational complexity and resource usage
- May generate false positive alerts in some situations

Relation to Proposed Project

This project utilizes a very similar approach whereby intelligent monitoring of threats and unauthorized access detection is incorporated into the design. Instead of deploying machine learning algorithms for this purpose, the project employs an artificial intelligence-based approach to security monitoring through the use of AWS CloudWatch, Lambda, auditing, and real-time alerting features.

2. Multi-Tenant Security Isolation in Cloud-Based Applications

Methodology

This paper highlights the various isolation methods that can be employed by multi-tenant cloud applications. The methodology will employ the use of tenant identification numbers, partitioning of databases, and access control based on identity. The methodology would mainly target SaaS applications whereby multiple users utilize the same resources safely.

Advantages

- Prevents unauthorized access and tenant data leakage
- Supports scalable cloud-based applications
- Allows centralized resource and infrastructure management

Limitations

- Managing access control policies can become complex
- Increases overall architectural complexity
- Shared infrastructure may still introduce certain security risks

Relation to Proposed Project

As with the suggested note management system, multi-tenancy will be achieved by creating a tenant isolation layer in DynamoDB. In addition, separation of data will be

achieved through proper database design that allows users' data to be stored independently through secure tenant isolation and authorization/authorization methods.

3. Serverless Security Architecture using AWS Lambda

Methodology

The security concerns regarding serverless computing based on AWS Lambda, API Gateway, and event-driven systems are discussed in this paper. This research emphasizes the advantages of serverless systems for reducing the effort required for infrastructure management and achieving flexibility and scalability in deploying applications.

Advantages

- Reduces the need for manual infrastructure management
- Supports automatic scaling based on workload
- Minimizes server maintenance efforts
- Provides cost-effective application deployment

Limitations

- May experience cold start latency issues
- Dependency-related vulnerabilities can affect security
- Limited execution environment compared to traditional servers

Relation to Proposed Project

The entire development of the note management application is serverless based. The AWS Lambda service will be used for the execution of back-end code and the logic of the application, while the AWS API Gateway will serve as the channel through which communication will occur between the user and cloud-based resources.

4. Cloud Audit Logging and Security Monitoring Systems

Methodology

The research study emphasizes on the auditing techniques applied to perform security surveillance in a cloud environment. The developed system continuously monitors the activities of applications and logs security data for security analysis. This allows the administrator to analyze any suspicious activity performed by the users in the cloud environment.

Advantages

- Improves security incident investigation

- Helps in tracking attacks and suspicious activities
- Supports real-time monitoring and log management

Limitations

- Requires large storage space for maintaining logs
- Log analysis can become complex in large systems

Relation to Proposed Project

It uses AWS cloud watch logs, aws dynamo db audit. Monitoring logs for user activity; possible security issues related logging access attempts as well lack of authorization on said activities. access attempts. This would improve protection, as well gain knowledge of anything that is not right. irregularities within the system. Such monitoring will enhance security and provide insight into any This would increase safety as well, giving us some idea of what happened to them at least.

5. Identity Spoofing and Shadow Identity Attacks in Cloud Systems

Methodology

this essay examines what role does a decoy user play when attacking someone else's system by impersonating them. Security controls as well as gaining entry for cloud computing services. This research will focalize your attention towards those who are targeted with acts of identitarian violence; i. with use of identification details or by getting access to cloud services with unusual activities.

Advantages

Increases awareness about modern identity-based attacks Fosters understanding of current cyber-attacks on identities. Emphasizes on securing cloud system's accessibility interface, rather than data itself.

Limitations

Promotes use for more robust identification methods. Advanced spoofing attacks are difficult to detect Sophisticated types of address forging techniques can be hard for detection. Needs constant observation of behavior and study.

Relation to Proposed Project

Basic security systems might not be able to stop complex attack patterns. The suggested solution mitigates the challenges relating to shadow identities and This suggestion alleviates some of those problems involving ghost people too. Unauthorized entry by way of authentication mechanisms, log trail audits. the supervision for unauthorised activities. It is always monitoring for any strange behavior.

6. Real-Time Threat Detection using Cloud Monitoring Services

Methodology

The study focuses on rapid identification of threats that occur within a cloud computing domain system. using a cloud native observability tool such as cloudwatch detect and respond to events automatically. generation, and alerting. the proposed approach requires regular tracking of alarms. processing event, raising an alarm if there's a problem detected.

Advantages

Facilitates tracking on-the-fly, live event activities within a cluster instance. Allows quicker detection of attacks. Supports automatic alerts and notifications systems.

Limitations

Needs to be properly configured with event triggers / alerts, etc. Alerts can be flooded due to too many of them.

Relation to Proposed Project

Suggested solution is to use amazon's cloudwatch, sqs, and sns service which can be used today itself. troubleshooting and alert for a system/security related issue. suspicious activity and/or unexplained contacts of mr. access attempts, anomalies etc. shall also continue to receive attention from us. be automatically generated.

7. Secure Data Encryption Techniques for Cloud Storage

Methodology

In this research paper, the study of techniques to provide security for data in the cloud has been carried out through AES encryption and hash algorithms. The approaches used in this paper ensure that sensitive data kept in cloud servers can be protected against any form of misuse through encryption.

Advantages

- Provides strong confidentiality and data protection
- Reduces the risk of unauthorized data access
- Supports secure storage of sensitive information in cloud environments

Limitations

- Encryption key management can become complex
- Encryption and decryption processes may introduce performance overhead

Relation to Proposed Project

The system follows secure storage practices and encrypted note handling techniques to improve data confidentiality and protect sensitive user information stored in the cloud

environment.

8. API Security in Cloud-Native Applications

Methodology

This research paper presents various approaches employed in securing API communication in cloud-native architectures. The research provides an analysis of various approaches such as token verification, authentication, authorization, and request filtering to ensure secure interactions between the end-user and backend services..

Advantages

- Prevents unauthorized access to APIs
- Improves secure communication between services
- Protects backend systems and cloud resources

Limitations

- Token and authentication management can become complex
- API misconfigurations may still create security risks

Relation to Proposed Project

For the suggested project, the role of AWS API Gateway in handling API traffic and validating requests in a secure manner is discussed. Security measures are employed in order to ensure that the APIs are accessed in a secure way, and the backend services are protected against any security breaches.

9. AI-Assisted Cybersecurity Monitoring Systems

Methodology

This research paper aims to provide an overview of the implementation of artificial intelligence technology for analyzing suspicious actions in cloud-based security monitoring systems. A cloud-based security monitoring system using AI is proposed and described in detail, including its ability to analyze user behavior, detect security anomalies, and potential attacks.

Advantages

- Enables faster detection of security threats and attacks
- Reduces the need for continuous manual monitoring
- Supports intelligent pattern recognition and automated analysis

Limitations

- Requires proper AI model training and datasets
- Detection accuracy may vary depending on system behavior and attack patterns

Relation to Proposed Project

The above-described project is also based on the same concept that employs AI-based threat detection through behavior tracking, detecting unauthorized access, creating audit logs, and analyzing security events. Even though machine learning algorithms have not been used for advanced modeling, intelligent surveillance methods have been applied for better monitoring of cloud security issues.

10. Secure SaaS Architecture for Multi-Tenant Applications**Methodology**

This paper introduces a safe and scalable SaaS architecture that can be used in multi-tenant cloud applications. The architecture is built based on tenant aware authentication, tenant database isolation, and security management. This way, each user will have safe access and separated data.

Advantages

- Provides high scalability for cloud applications
- Supports efficient tenant and resource management
- Improves resource sharing within a shared cloud environment

Limitations

- Tenant migration can become difficult in complex systems
- Shared platform infrastructure may still introduce security vulnerabilities

Relation to Proposed Project

The suggested notes management software is designed on a similar multi-tenant architecture inspired by the concept of security as a service (SaaS), leveraging the capabilities of the AWS Cloud platform.

2.2 Gap Analysis

Current cloud-based note management systems only focus on core functions that allow users to create, edit, delete, and store notes. Additionally, they offer rudimentary user authentication facilities. However, the majority of classic applications lack sophisticated security measures, including tracking unauthorized access attempts, real-time monitoring, audit logging, artificial intelligence-powered intrusion detection, and adequate tenant isolation.

At present most of them are working only by means of passwords for authorization purposes. ignore user's actions, hide-user-identity assaults. So criminals may be able to execute. numerous suspicious login events detected and blocked. Moreover, most systems ignore security alerts as well as logs of activities. ignore warning messages for insecurity, do not record activities off line.

In fact, according to researches, there are many cloud security models already proposed which concentrate more on theoretical aspects of intrusion detections than on practical approaches to the implementation of security solutions within a cloud infrastructure. These solutions do not make use of up-to-date AWS tools like AWS Lambda, API Gateway, CloudWatch, SNS, and DynamoDB for monitoring and analytics purposes.

Another drawback associated with the usage of some security solutions is high complexity as they require machine learning models to be trained on massive amounts of data.

These shortcomings have been addressed in the proposed project through the use of serverless cloud architecture, real-time monitoring, audit logging, and AI-inspired threat detection, among others, in a simplified manner. The proposed project resolves the limitations of cloud-based note management and security solutions by creating an effective serverless cloud security framework that incorporates AI-inspired threat monitoring capabilities. In contrast to most applications that only concentrate on providing the basic functionality, the proposed solution is enhanced with security features alongside the scalability aspect of cloud-native architecture.

The application comes with various capabilities, including secure multi-tenancy architecture, unauthorized access attempts logging, real-time security notifications, CloudWatch monitoring, auditing, shadow identity identification, AI-assisted threat intelligence, and serverless scalability, among others.

Among the other innovations that can be highlighted in this project is the use of intelligent security monitoring in conjunction with the serverless cloud architecture in the context of lightweight notes management application. This system leverages cloud-native AWS solutions and incorporates multi-tenancy and real-time monitoring for enhancing the efficiency of both applications scaling and cyber security management.

3 Proposed Methodology

Introduction

The design of the new system includes the use of a secure serverless cloud architecture that will allow note management in addition to threat detection and monitoring through the help of AI techniques. Several cloud-based services provided by AWS such as API Gateway, AWS Lambda, DynamoDB, CloudWatch, and SNS are used together in the creation of an application based on this architecture.

The first objective behind choosing this architectural design is to allow for secure data management and security monitoring of threats within the application. The design makes sure that secure tenant isolation is ensured so that each user can access his/her data alone without having access to other users' information. Threats can be monitored real-time within the application, and unauthorized access detected easily.

This application is based on an architecture that has several layers, in which each AWS service has its own task in processing, storing, monitoring, and security management. AWS Lambda takes care of backend processing and executing the business logic. The API Gateway deals with the secure APIs, DynamoDB is used for securing user notes, CloudWatch deals with logging and monitoring, while SNS sends out security alerts.

This architecture utilizes serverless computing, cloud security, and AI-driven threat detection to develop a lightweight and efficient secure notes management system.

1. Frontend Layer

Frontend Layer is the user interface of the application through which users are able to interact with the system effortlessly. Using the frontend, users will be able to create, update, delete and also view the notes that they have created securely. Through this interface, users are also able to monitor any security alerts and monitor attempts of unauthorized access within the application.

The frontend for the system is created using HTML, CSS and JavaScript. These technologies assist in developing a responsive and user friendly interface for the application.

Responsibilities of the Frontend Layer

- Managing user interaction with the application
- Sending API requests to backend services
- Displaying notes and security-related information

- Showing real-time security alerts and monitoring updates
- Updating dashboards and security metrics dynamically

The frontend layer plays an important role in connecting users with the cloud-based backend system while providing a smooth and secure user experience.

2. API Gateway Layer

API gateway's function as an intermediary for communications between client and server, specifically related to user interfaces (uis). throughout our back end clouds. The AWS apis gateway would help us out for everything here.

Transactions of API to app server side for end users. If an entry is created, edited deleted viewed then it will have to go through this process initially. to the API Gateway. After that, it can go for lambda function pertaining this topic.

Responsibilities of the API Gateway Layer

- Functions of the api gate way or tier layers.
- Handling API request routing
- Controlling http verbs like get, post, put and delete.
- Validating incoming requests
- Ensuring safe call to an api end point.
- Merging front-end calls to aws lambda function.
- Advantages of API Gateway
- Provides secure API management
- Fosters flexible interaction among components.
- Manage, as well monitor new demands.

Facilitates safe communication among front-end servers, back-ends systems. Aids to effectively administer and handle clouds-based applications traffics adeptly. API gateway is a critical part of achieving safe, trustworthy connections to services within your system. in a server-less computing model.

3. AWS Lambda Processing Layer

the lambda function does everything behind your back while you're doing something else with aws. application. The application runs on lambda servers since it is based upon server less model. automate handling of requests instead off managing them on your own. records are modified when creating/updating/deleting/reading records by users. calls would go to lambda's. it uses lambda as well to do stuff. risk management security breach assessment.

Main Lambda Functions

1. NotesHandler Lambda – performs all tasks related to notes like creation and modification.
NotesHandler Lambda - Handles all note-related operations such as creating, updating
2. SecurityMetrics Lambda monitors security events, unauthorized access attempts, and threats.
3. suspect behavior of a program. SecurityMetrics Lambda - Monitors security events, unauthorized access attempts, and
4. suspicious activities within the application

Responsibilities of the AWS Lambda Layer

- Features of the amazon web services (AWS) Lambda container Layer.
- Performing cru actions on a note.
- Executing a person's authentication as well identification verification procedure.
- Keeping log files and trail information up-to-date.
- Detecting unauthorized access attempts
- Monitoring security-related events
- Promoting artificial intelligence-based risk assessment, behavioral monitoring capabilities.
- Benefits of AWS Lambda
- Provides fully serverless execution
- Offers auto-scaling according to app load.

- Decreases facility administration and upkeep expenditures.
- Improves deployment speed and flexibility
- Support to create low-cost cloud native apps.

AWS lambda is important for handling app load and secure it. tracking, and back-end functions effectively as per this new design of a platform.

4. DynamoDB Data Layer

Dynamo db data layer serves to store and manage your application's database securely. manner within the cloud. DynamoDb from Amazon is a fully-managed nosql dbaas. provided by aws that guarantees good speed as well as flexibility of use. This is managed by our service called dynamo db and includes storing your comments, authorised people who can access them as well as logins. reviews, and a record on failed access logs. The system has been designed for securing this data base. data multi tenancy, ha, fast-data-access. Database Tables

Database Tables

- NotesTable – Stores all user notes and related note information
- ApprovedUsers – Maintains records of authorized users and access details
- SecurityAuditLogs – Stores security events, suspicious activities, and unauthorized access attempts

Security Features of DynamoDB Layer

- Supports secure tenant isolation between users
- Provides fast and efficient access control mechanisms
- Ensures secure NoSQL cloud-based data storage
- Helps maintain scalable and reliable application performance
- Supports real-time data access for monitoring and security analysis

The DynamoDB layer plays an important role in securely storing application data and supporting the multi-tenant architecture of the proposed system.

5. CloudWatch Monitoring Layer

CloudWatch layer is responsible for monitoring the applications activities and security events taking place on the system. AWS CloudWatch is employed for collecting logs, monitoring system performance, and monitoring any suspicious activities that may be going on in the system.

In our case, CloudWatch will monitor Lambda logs, unauthorized accesses, security events, any runtime errors, and audits.

CloudWatch Monitoring Includes

- CloudWatch Monitoring Includes
- Tracking lambda functions log files.
- Monitoring incidents, anomalies and threats to system safety.
- Detecting unauthorized access attempts
- Detecting run-time bugs, crashes and faults.
- Keeping records of audits to aid with reviews and assessments.
- Advantages of CloudWatch Monitoring
- Monitors applications live time to life.
- Facilitates central logging data monitoring and administration.
- Support for monitoring of security and system events.
- Enhances insight on risky activities, dangers.

Helps with quicker identification of, as well as reaction time for cybersecurity events. Cloudwatch monitor is very useful for a enhancing your view of information and safety issues. ensuring constant surveillance of a cloud native app architecture.

6. SNS Alert System

SNS can be utilized for triggering automated security alerts when certain conditions are met. Any unusual activities and intrusions that are not authorized. Amazon Simple Notification Service helps to notify via a platform as well increases safety levels. Herein lies that we would use mainly to notify of all kind of incidents including login attempts etc. failure, suspicious activities, and strange behavior detected by the police. system. **SNS alert system duties.**

- Alerting on failed login attempt detection.
- Generating security warning alerts
- Reporting on threats live and as they occur.
- Automated security response mechanisms support.
- Benefits of SNS Alerts
- Supports real-time alerts and notifications.
- Improves quick reaction time for threats and vulnerabilities.
- Enhances general safety consciousness as well as surveillance capabilities.
- Facilitates effective reporting on threats incidents.
- Improves insight of malicious processes on systems.

The SNS alarm tier is crucial for enhancing current time risk identification. making sure prompt reaction on possible threats of a software program that is hosted online.

System Architecture Explanation

System design starts with a customer utilizing an app's front end for performing some action. tasks such as inserting, updating, erasing and reading from a diary. Those front-end demands. securely pass through the API Gateway of Amazon Web Services (AWS), which functions as a. boundary of UI with back end service. Then, it sends those calls on their way back through an appropriate amazon web services (AWS) lambda processor. The back-end operations authenticate a user's identity as well performs all of his/her tasks regarding their journals. Prior to executing the tasks requested by the user, the system verifies that the user has valid permissions.

All application data, from user comments, authorized user details, audit logs to unauthorized access details, are stored in Amazon DynamoDB. The database provides secure multi-tenant capabilities for storage and guarantees that only authorized users can access their own data.

AWS CloudWatch offers continuous monitoring for Lambda execution logs, security incidents, runtimes and any other suspicious activity on the application. It assists in monitoring any unauthorized access attempts to the application.

In case there is any suspicious activity on the system, AWS SNS sends automatic alerts regarding such activity. This increases security awareness among the administrators.

The architecture also assists with AI-driven threat detection through the use of monitoring the behavior patterns of users, detecting suspicious transactions, and analyzing any attempts at gaining unauthorized access.

3.1 System Architecture / Block Diagram

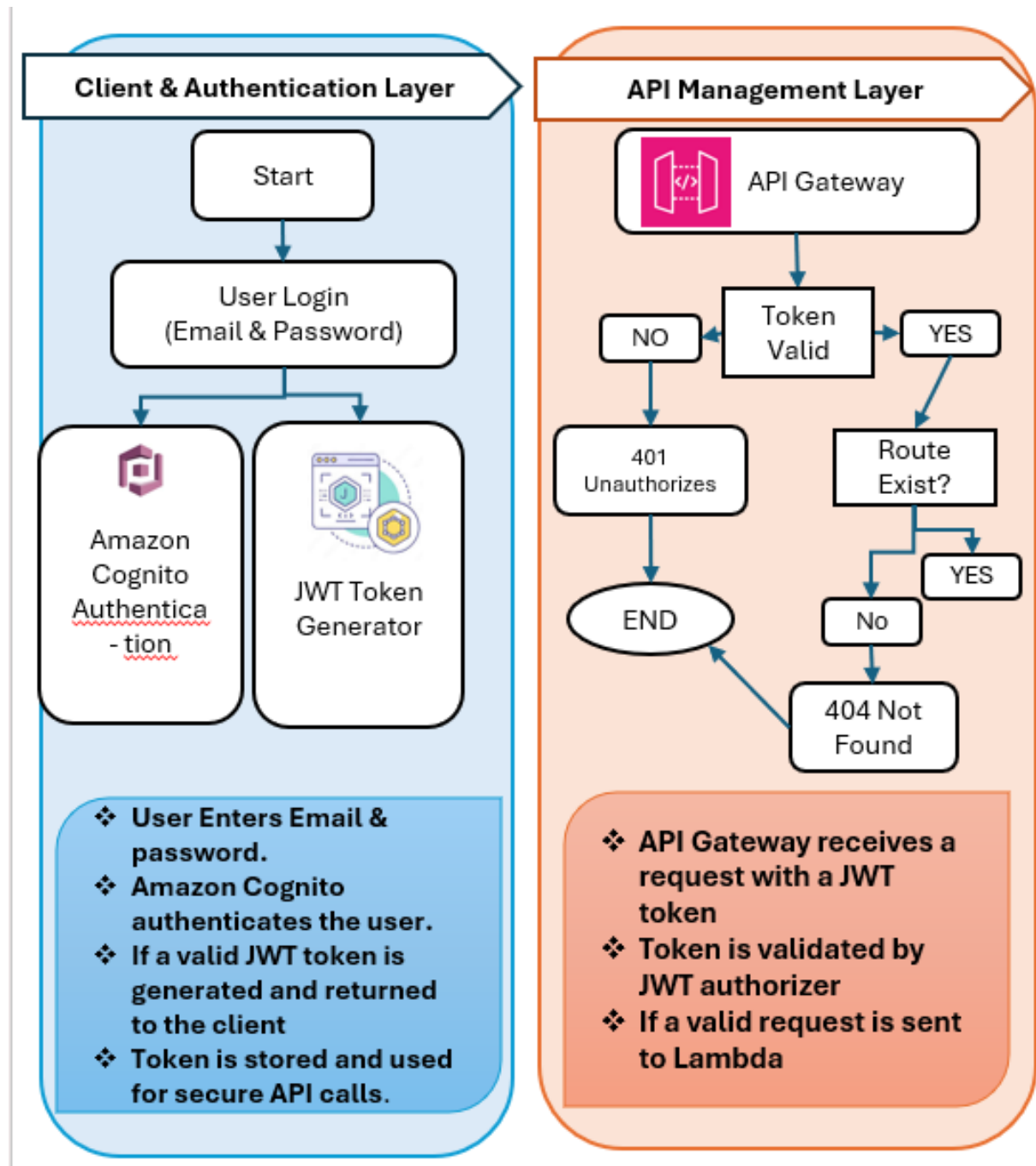


Figure 3.1: System Architecture

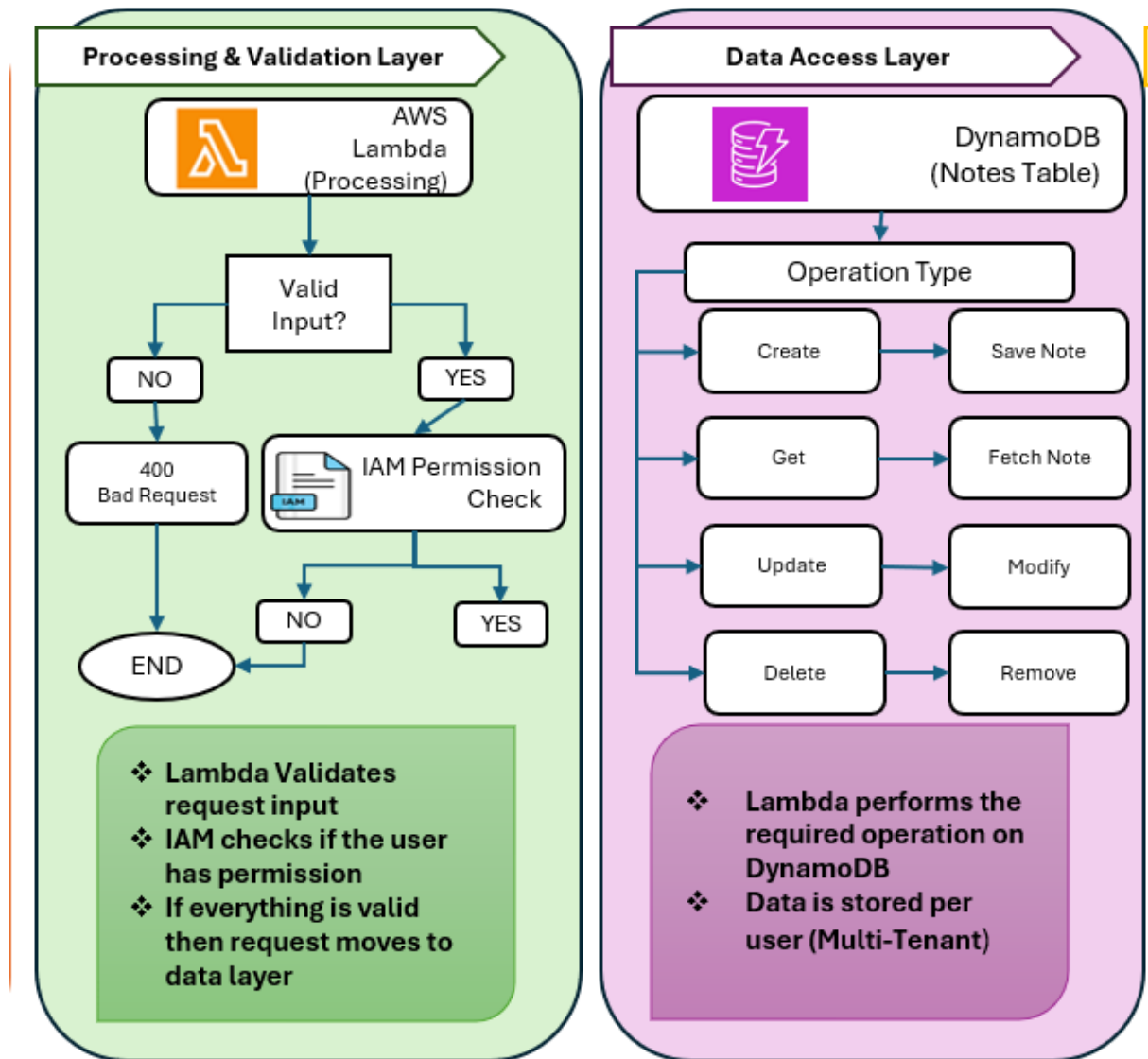


Figure 3.2: System Architecture

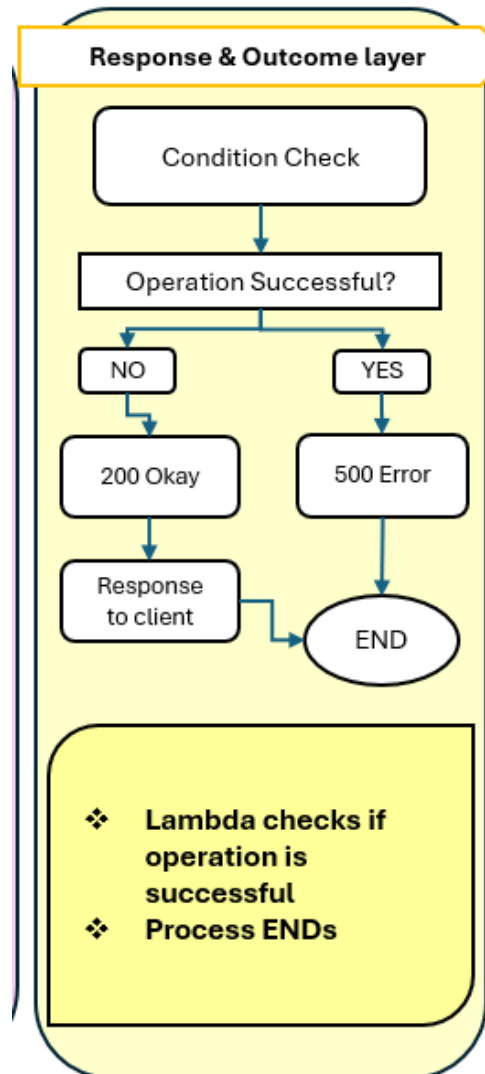


Figure 3.3: System Architecture

3.2 Proposed Methodology

The methodology under consideration intends to design a highly secure cloud-native application, which would be capable of protecting the tenant data from any kind of vulnerabilities. Furthermore, the system would monitor any abnormal behavior within the platform in real-time.

The methodology seeks to ensure that the system is able to provide secure note management, tenant-based isolation of data, unauthorized user access detection, and overall monitoring of the security aspects through the use of AWS cloud-based services. Also, the app is going to follow server-less model so that less human interaction can happen along with easy scalability. To successfully implement this solution we divide it up through several stages such as front-end design, APIS connection, back end activities, database maintenance, auditing logs, troubleshooting alerts etc.

The proposed methodology helps in creating a lightweight, scalable, and secure cloud-native application capable of handling real-time monitoring and threat detection in multi-tenant cloud environments.

Phase 1: Requirement Analysis

The first stage of the project included analyzing the main security threats and requirements that arise in the field of cloud-based multi-tenant applications. As part of this project, attention was primarily paid to the analysis of the most frequent security threats in cloud applications and an assessment of the threats posed by keeping users' sensitive information in the cloud.

As a result of the initial stage of the project, it was possible to identify such major security threats as unauthorized access, identity spoofing, shadow identity attacks, insufficient audit monitoring, and other risks associated with shared tenant infrastructure. Such security threats can cause data leakage and suspicious access requests from different sources.

Based on the security requirements identified during the initial stage of the project, its objectives were formulated.

Phase 2: System Design

This step was to design an effective server-less architecture of the note solutions. By using Amazon Web Services it should have been a scaleable and provide for safe communication. centralize monitoring spot security issues more efficiently and streamline your it operations with aws. it should have been scaleable and provide for encrypted communication. central monitoring, effective threat detection, simplified it infrastructures.

Designed Components

- Frontend Dashboard
- AWS API Gateway
- AWS Lambda Backend
- DynamoDB Database Tables
- CloudWatch Monitoring System
- SNS Notification Service
- Security Objectives
- Maintaining secure tenant isolation
- Providing secure API communication
- Surveillance of illicit actions and dangers.
- Keeping trail files and secure data entries.
- Real time support for alerts creation and maintenance.

The system design phase enabled an organized cloud-native architecture to be created. serving as a support for applications' performance, along with sophisticated tools that monitor their safety levels.

Phase 3: Multi-Tenant Database Design

for that phase we created architectural design of multitenant db on aws. DynamoDB framework. This design allows for many people to use it at once. they could also potentially get saved to a db, but need some isolation between each other. **Features**

- User specific space to store tenant data.
- Data partitioning based access control.
- Secure note retrieval mechanisms
- Data source authentication prior to accessing it.

Benefits

- precludes unintended inter tenant information sharing.
- Enhances applicability of scale-up, as well as efficiency.
- Promotes safe, fast data archiving on a server farm.
- Ensures appropriate segregation of users' data.

A multi-tenant model allows users to view and edit data they are authorized for, not others' entries. And store them safely on this platform too.

Phase 4: Authentication and Access Control

Authentication, authorization and accounting (AAA) mechanisms are implemented during that stage for security purposes. The safety and protection against unauthorized use, which may lead towards a threat attack on it. The system identifies users before providing them with service on an app base system. management processes.

Security Techniques

- User Authorization Verification
- Access Control Mechanisms Based on Email
- Refusal of Access to Unauthorized Users
- Authorization/Permission Logic Validation Processes

Innovation Aspect

One other significant development is due to its introduction, which involves tracking login hacking attempts as well monitoring to keep records of such attacks. This ensures any unusual activities are recorded for analysis so as improve security levels on clouds. application security.

Phase 5: AI-Inspired Threat Detection

This is the stage where security monitoring features inspired by AI were implemented to identify any suspicious activities. The application will monitor both user actions and security events in order to detect anomalies in their pattern of behavior.

Threat Detection Features

- Unauthorised access attempt tracking
- Suspicious identity monitoring

- Unusual user activities detection
- Event pattern and security behavior analysis

Why AI-Inspired?

Rather than relying on sophisticated machine learning algorithms and extensive amounts of data for training, the current effort makes use of simple intelligence for monitoring to detect any suspicious activity. This makes the system less complex but is still capable of providing sufficient security monitoring functions.

The cloud application security monitoring tool inspired by AI assists in improving security awareness.

Phase 6: Security Logging and Monitoring

In this phase, CloudWatch monitoring and audit logging capabilities from AWS have been utilized to achieve continuous monitoring of activity in applications and any security incidents. Logging is used to document any crucial events that occur.

Logged Events

- User successful login
- Unsuccessful unauthorized login attempts
- User CRUD actions performed
- Security incidents and suspicious events
- Application run-time issues

Advantages

- Enhances understanding of security threats
- Assistance in detection and monitoring of suspicious events
- Facilitates security audits and forensic investigation
- Centralized monitoring and logging capability
- Helps detect abnormal activities
-

The above phase is significant in improving the security of applications through various means including monitoring, logging, and analysis of security threats.

Phase 7: SNS Alert Integration

For this step, AWS SNS (Simple Notification Service) was added to the application for sending automatic security alerts every time suspicious activity or a security threat is discovered. This improves security monitoring and makes it possible to respond to security threats effectively.

Security Alert Events

- Unauthorized access
- Suspicious activities related to security
- Automatic security alerts created through the monitoring system

Advantages

- Responds immediately to any security issues
- Increases awareness of security within the system
- Enables real-time alerts
- Facilitates monitoring of suspicious activities

This enhances the entire security framework by providing instant communication on security events.

Phase 8: Testing and Validation

In the final stage, the entire system was tested to check for proper working, security, and performance of the system. Various test scenarios were used to confirm both the operations of the application as well as its security monitoring functionalities.

Tests Conducted

- Tests by authenticated users
- Tests by unauthenticated users
- Verification of CRUD functionalities
- Test of security events
- Real-time monitoring tests

It was seen that the system was able to properly detect any unauthorized access attempts, create tenant isolation in a secure manner, and send out real-time security alerts. It could also perform note management tasks securely in the serverless cloud computing infrastructure.

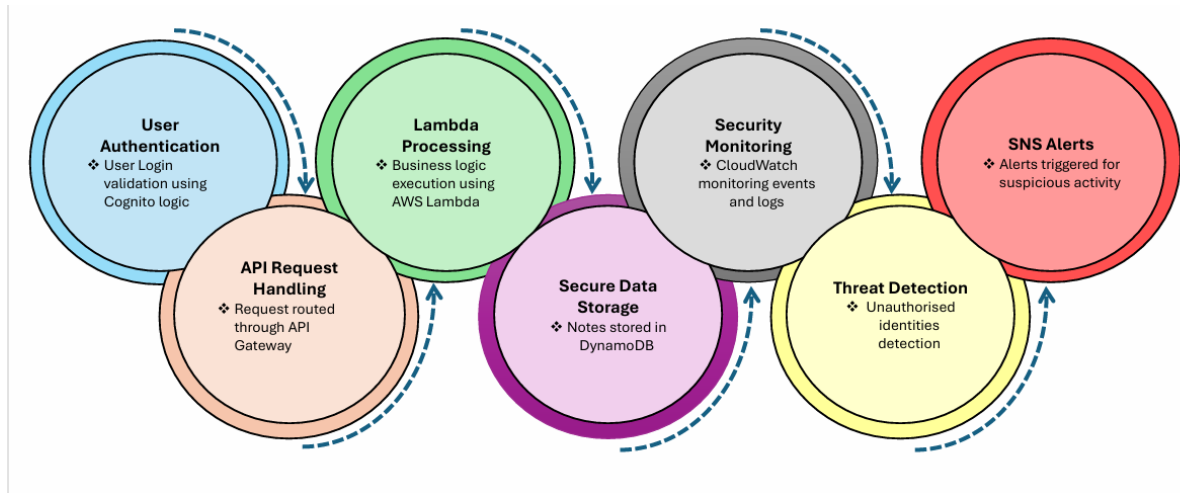


Figure 3.4: Methodology

3.3 Implementation(Development and Deployment procedures)

Step-by-Step Development Procedure

Step 1: Frontend Development

In the first stage, the frontend dashboard of the application was built using common web development techniques. The HTML language was used to build the application layout, CSS was used to style and design the frontend, while JavaScript was used for API interaction.

The frontend was created to provide an easy-to-use interface for handling notes and keeping track of security-related events.

Frontend Features Implemented

- Dashboard for managing notes
- Display of security metrics
- Display of unauthorized access attempts
- Interface for performing CRUD operations on notes
- Interaction with backend services in real-time

The frontend serves as the channel of communication between users and the backend services hosted in the cloud.

Step 2: AWS Lambda Development

Lambda functions are designed at stage 2, which is used as a back-end processor. execution of business logic. Since this application is built on aws lambda, the lambda function has been used. it would handle it on its own, no intervention from a webserver is needed 24/7. developers.

Lambda Functions Created For

Making, editing, erasing, finding out information from your diary. Validating user accounts and verifying access rights. Tracking safety measures, keeping track of audits records. AI for threat analysis, suspicious behavior detection.

Step 3: DynamoDB Configuration

At present, we are setting up aws dynamo db to be our primary cloud database for storing data. the programs are going to stay safe here. Multi tenant is possible with this db architecture. audit storage and secure abilities. DynamoDb has been established like a primary aws cloud database that holds all our data. your information is safe here. Multi tenancy is supported by this db architecture.

Tables on DynamoDB Created

- NotesTable - notestable are used to store users personal data
- ApprovedUsers - Storing data about authorized users
- SecurityAuditLogs - Storing logs of audits, any failed accesses, and security incidents

The setup allows for secure data storage, tenant isolation, and quick access to application data in the cloud.

Step 4: API Gateway Integration

This is the stage where AWS API Gateway was combined with the application so as to facilitate the safe exchange of information between the frontend and the backend. REST APIs were set up to cater to various user requests and link them to respective AWS Lambda functions.

REST APIs Configured

- GET Request – For fetching notes and security information
- POST Request – For creating notes and posting information
- PUT Request – For updating information on notes
- DELETE Request – For deleting notes from the database

API Gateway ensures safe routing and communication between API and the serverless environment.

Step 5: CloudWatch Monitoring

In this stage, the AWS CloudWatch was set up for continuous monitoring of the application and any security activity.

The following aspects were included in the configuration of CloudWatch.

- Monitoring of Lambda function runtime
- Logging and centralized monitoring
- Error detection and failure analysis
- Visibility of threats and suspicious activities

This monitoring system can help improve security visibility and detect unusual behavior.

Step 6: SNS Alert Integration

The AWS SNS service was implemented in order to ensure that there is an automatic process for notifications for security related incidents that were discovered in the app. This ensures improved response capability through automatic alerts whenever suspicious incidents happen.

SNS Topics

- Security Alert Notifications
- Notifications about unauthorized access
- Threat Detection Alerts

Step 7: Security Testing

During this last phase, testing for security was done to check the effectiveness of the security features of the application. Tests were conducted to ensure that the operations were safe and could be monitored.

Test Features

- Login unauthorized test
- Isolation of tenancy test
- Validation of security events monitoring
- Testing of CRUD operations

The tests demonstrated that the application had successfully provided tenant isolation, detected any malicious activity, and generated security alerts in real time.

3.3.1 Flow Diagrams

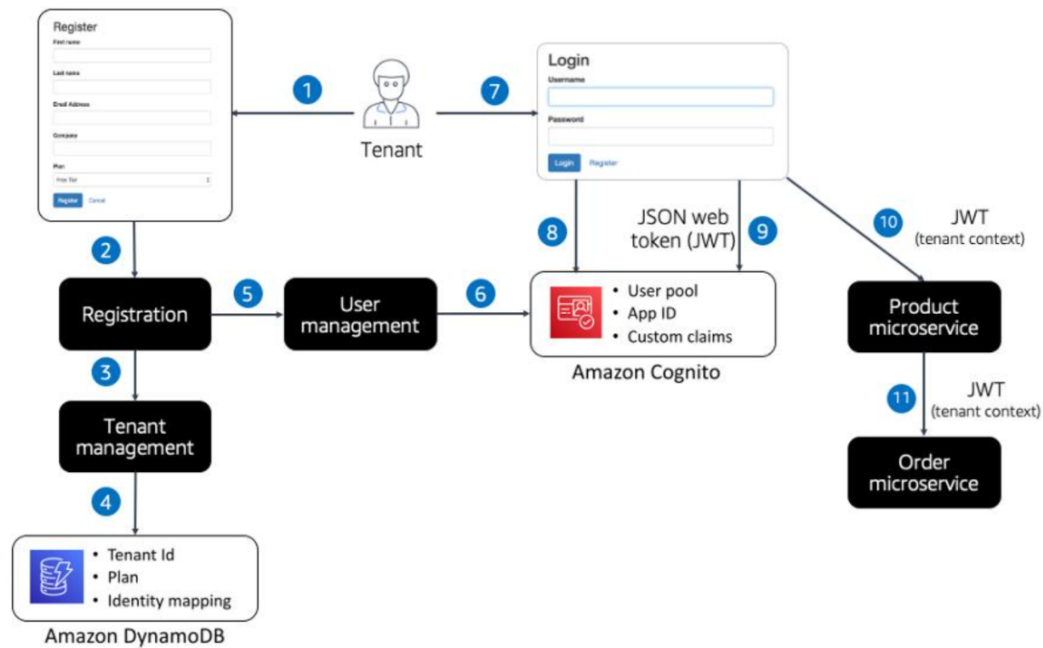


Figure 3.5: Flow Diagram

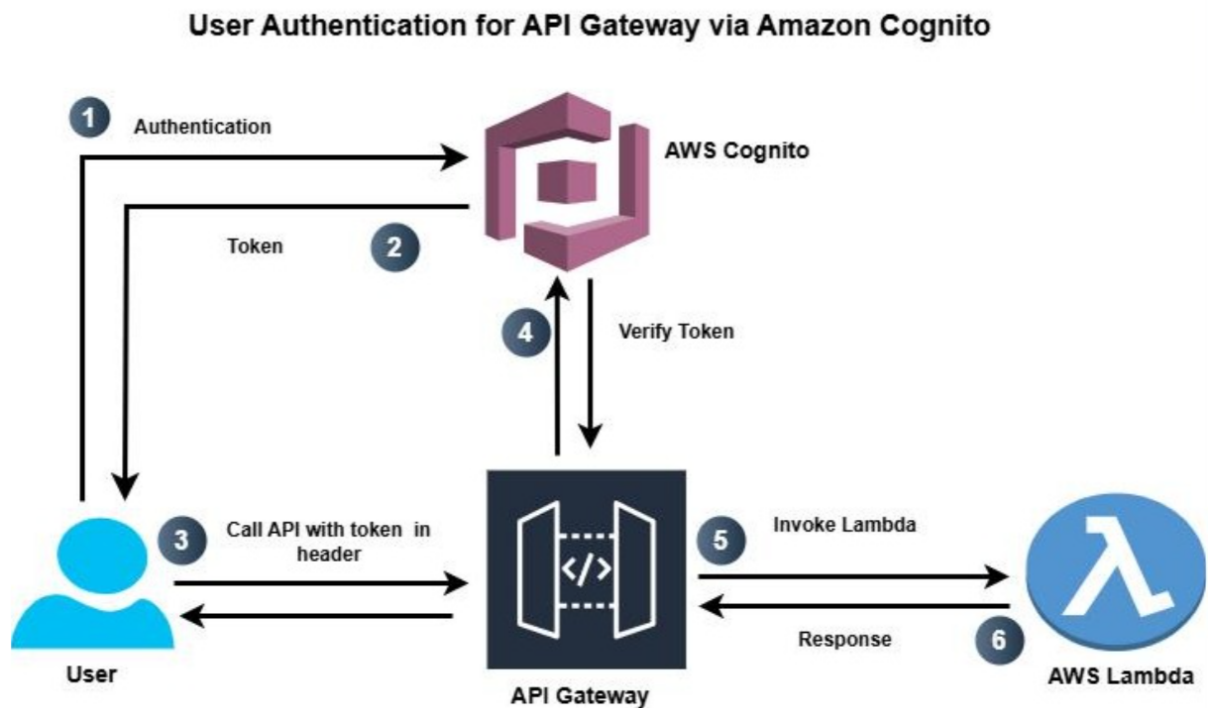


Figure 3.6: Flow Diagram

4 Result and Discussion

Multiple tests of this created software were conducted for various working, as well as safety situations so it could work properly while performing normally and abnormally. While implementing various obstacles to adopting a cloud platform arose such as apis connectivity problem, permissions setting for aws lambda function was confusing. The problems have been properly solved by troubleshooting logs as well as sla reporting.

Our product has been extensively examined several occasions for a variety of operating as well as safety situations so it is able to function properly within regular plus emergency circumstances. Implementation revealed some of these real-life aws cloud deployment challenges like integration with apis, Lambda permissions setting up problem. The problems are well solved by means of debuggings, logs inspection as also level-of-service surveillance.

It was shown that centralised management is essential for cloud-based projects today as well. While conducting tests, cloud watch log was constantly being referred for identifying back end server problems like unauthorised login attempt s. If you don't monitor availability then it is very hard to debug a cloud application since there are no servers available for inspection with server-less architecture.

An additional finding of our work is a reduction in system intricacy due to implementing serverless cloud computing model. AWS lambda did not require manual configuration for back-end server, os as well. It increased productivity, streamlined operations for applications 24/7.

It also showed again that a cloud alert system improves your ability to respond after an event has occurred. Social network system alerts will notify managers of any criminal activity as soon it happens. The tool is very helpful for detecting threats on enterprises' clouds as soon possible. It was helpful to understand what keeps tenants isolated within a true cloud service like software as a service (sas). Validating tenants' identities prior to each and any db transaction ensures that no unwanted transnational information sharing takes place between them. In general, the test outcomes showed that this design could be successfully used to protect several people simultaneously with a possibility of watching what is going on.

4.1 Implementation Approaches

Secure multi-tenant notes application is built on modularity of cloud native architecture with services from amazon web services performing their specific role for security and processing. Implementation included front-end coding, back end api integrations, database setup, user authentication checks, monitoring configurations, and threats detections.

HTML, CSS and javascript were used while developing front end of web application which gives user friendly as well safe ui/ux experience. It enables you to develop, modify / edit/ erase; view safety occurrences as well like intrusion attempt.

AWS lambda function were used for implementing back end code which processes apis calls on cloud platform (server-less). The API gateway is employed for secure routing of front-end traffic towards aws lambdas. It simplified infrastructures, making them more flexible for scaling requests throughput as well.

The dynamo db is my main no sql data store to hold users' entries, tenants etc. To maintain tenant separation, each record is linked exclusively to one client's id so that no other tenants can view their records inadvertently. The cloudwatch service has been deployed for monitoring of security events, logs at runtime unauthorized access attempts as well system activity. The security log of every lambda function execution was monitored for any unusual activity detected.

The SNS service is set up so that it sends out notifications when there are any significant actions related to cybersecurity and suspicious activity detection occurs automatically. It includes an artificial intelligence-based algorithm for detecting threats, which monitors specific behaviors like excessive failed login trials and unusual requests from users.

Secure multi-tenant notes application has been implemented with modularity, native to cloud services wherein each amazon web services (AWS) component is responsible for its own operations as well as security related tasks respectively. It was about developing an agile, flexible, safe, and accessible cloud-based infrastructure for running applications that could be used by many people without compromising their privacy. It is split up to have separate parts which can all work independently of one another, be built by themselves, tried out on their own etc. The modularity approach enhanced adaptability of systems as well as their ease for adopting to clouds.

It is subdivided as follows, The project has been broken up to three main stages.

- Frontend dashboard development

- Backend API development
- AWS cloud service configuration
- Multi-tenant security implementation
- Unauthorized access monitoring setup
- Cloud logging integration
- SNS alert configuration
- Security event visualization
- Testing and validation

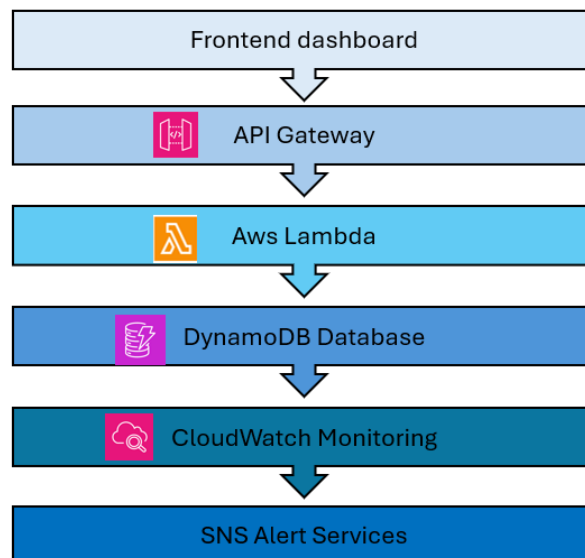


Figure 4.1: Implementation Approaches

4.1.1 Overview of the Implementation Strategy

Implementation approach included creating an efficient, safe server-less cloud solution that would support multitenancy as well as provide for security monitoring capabilities.

It is broken up as follows The work will be carried out through three stages.

1. Frontend dashboard development
2. AWS cloud service configuration
3. Lambda backend
4. DynamoDB table creation
5. API Gateway configuration
6. CloudWatch monitoring
7. SNS alert
8. Security testing and validation

Every phase of each module had to be tried out separately prior to combining them together as an overall working program suite.

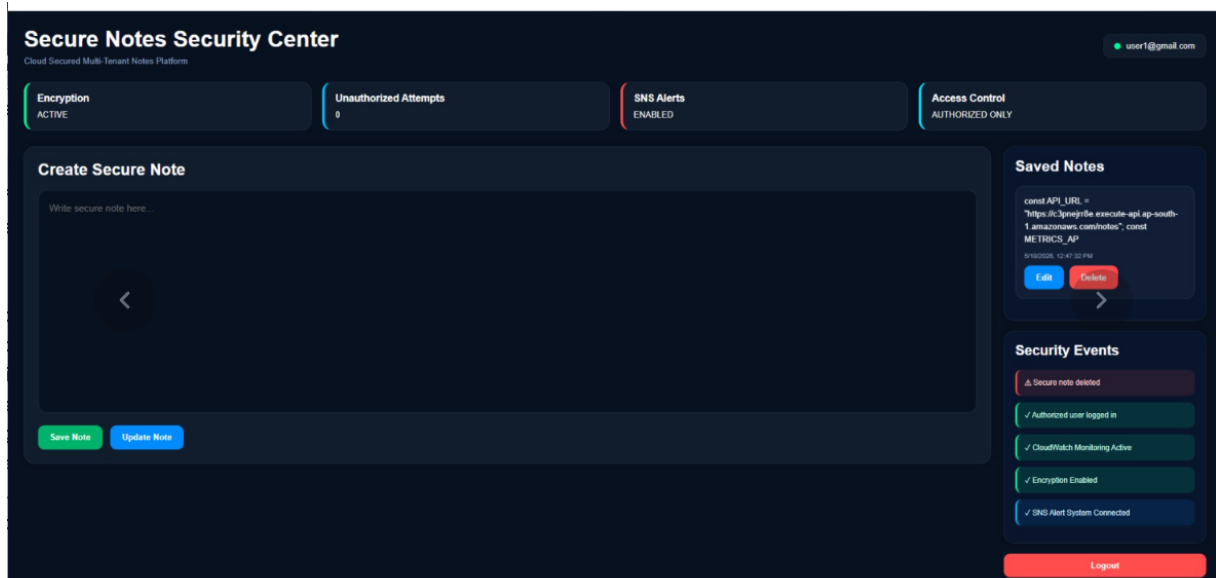


Figure 4.2: Overall implementation strategy

4.1.2 Environment Setup and Technology Stack

AWS cloud infrastructure has been set up for this project environment. is deployed on aws cloud platform. it was set up on aws clouds.

Technologies Used		Purpose
HTML/CSS/JavaScript	➡	Frontend UI
AWS Lambda	➡	Backend Processing
API Gateway	➡	Secure API Routing
DynamoDB	➡	NoSQL note storage
CloudWatch	➡	Monitoring and Logs
SNS	➡	Email Security Alerts
IAM	➡	Access control
VS Code	➡	Development environment
GitHub	➡	Version Control

Figure 4.3: Environment Setup and Technology Stack

4.1.3 Multi-Tenant Data Isolation Implementation

Tenant isolation was one of our key innovations on this platform. Each item saved to dynamodb has an owner's id attached. During every API the request is verified by a logged-in account and then granted permission for editing notes. That program blocks access to other tenants' secrets by a single user. information. Still even with an illegitimate command created by hand there's no denial of service check. blocks cross-tenant operations. Isolation boosts cloud security and mimics real world saas apps. enterprise platform security architecture systems and models.

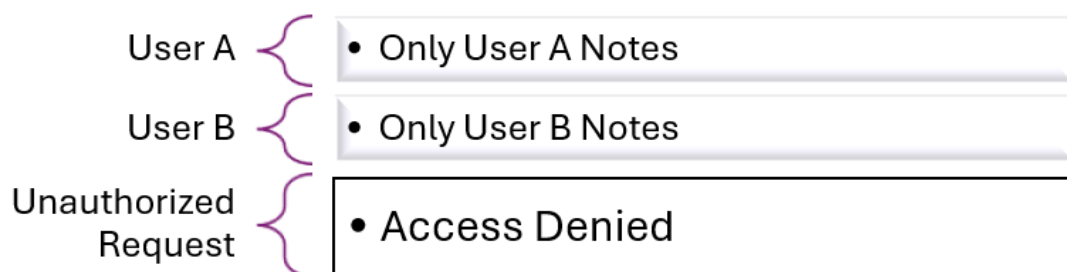


Figure 4.4: Multi-Tenant Data Isolation Implementation

4.1.4 Unauthorized Access Detection

Our solution is designed to monitor for unauthorised intrusion online live time. If a bad guy tries logging on then his attempt gets denied and he's recorded by aws cloudwatch logs. It increments to unknown source count, creates a monitor event. The tool increases awareness of potential threats by enabling admins to detect patterns indicative of repetitive harmful logins. Unauthorized access attempts tracking is a form of artificial intelligence inspired security system for this work.

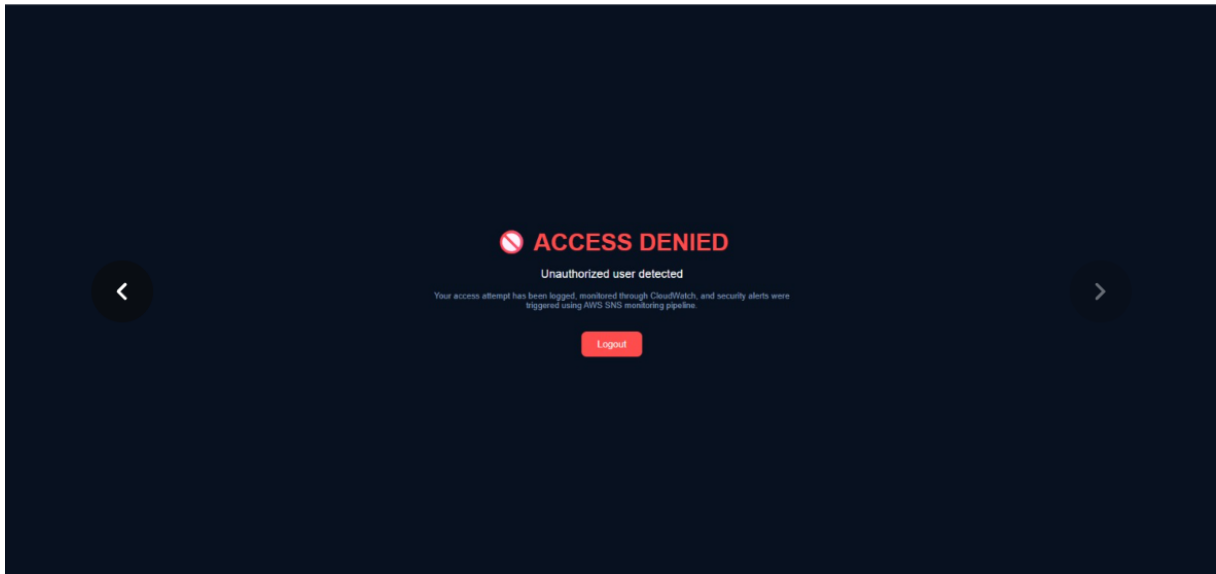


Figure 4.5: Unauthorized Access Detection

4.2 Pseudo Code

A pseudo-code represents an abstract structure and logic behind any computer program, application etc. Helps to understand how an algorithm works prior to coding it on any programming language. For our task we have developed pseudocode of all significant modules such as user login verification system, data security while storing notes, user unauthorised usage identification, AI based threats recognition mechanism, detection algorithm on a fake account creation, billing and surveillance management solution etc. Pseudo coding helps us specify how it would execute, how does data gets secured while accessing this service from a specific tenant on cloud. This work uses pseudocode to achieve these main goals. User authentication and authorization Secure CRUD operations Tenant-based isolation Machine learning based suspicion of fraudetection technique. Unauthorized access monitoring CloudWatch log analysis SNS alert generation Security event handling Pseudo-codes have been written to implement most components of this project.

Pseudo Code for User Authentication

```
START
INPUT user email
INPUT password

CHECK email in AuthorizedUsers table

IF email exists THEN
    VERIFY password
    IF password is correct THEN
        ALLOW user login
        GENERATE access token
        DISPLAY dashboard
    ELSE
        STORE unauthorized attempt
        SEND SNS security alert
        DENY access
    ENDIF
ELSE
    STORE shadow identity attempt
    SEND security notification
    DENY access
ENDIF
END
```

Figure 4.6: Pseudo Code for User Authentication

Pseudo Code for Create Secure Note

```
START
USER enters secure note
VALIDATE user token
IF token valid THEN
    ENCRYPT note data
    GENERATE unique note ID
    STORE note in DynamoDB
    LOG security event
    RETURN success response
ELSE
    DENY request
ENDIF
END
```

Figure 4.7: Pseudo Code for Create Secure Note

Pseudo Code for Fetch Notes

```
START
RECEIVE GET request
VERIFY authentication token
IF token valid THEN
    FETCH notes from NotesTable
    FILTER notes using tenant ID
    RETURN notes to user
ELSE
    RETURN unauthorized response
ENDIF
END
```

Figure 4.8: Pseudo Code for Fetch Note

Pseudo Code for Update Notes

```
|  
START  
  
RECEIVE updated note data  
  
VALIDATE authentication  
  
CHECK note ownership  
  
IF authorized THEN  
    UPDATE encrypted note  
    STORE updated timestamp  
    RETURN success message  
ELSE  
    RETURN access denied  
ENDIF  
  
END
```

Figure 4.9: Pseudo Code for Update Notes

Pseudo Code for Delete Notes

```
|  
START  
  
RECEIVE delete request  
  
VALIDATE user token  
  
CHECK note ownership  
  
IF authorized THEN  
    DELETE note from database  
    LOG delete event  
    RETURN success response  
ELSE  
    RETURN unauthorized response  
ENDIF  
  
END
```

Figure 4.10: Pseudo Code for Delete Notes

Pseudo Code for Unauthorized Attempt Counter

```
START  
  
RECEIVE login request  
  
CHECK user credentials  
  
IF credentials invalid THEN  
    INCREMENT unauthorized attempt counter  
    STORE attempt in SecurityAuditLogs table  
    SEND SNS alert notification  
    UPDATE dashboard metric  
ELSE  
    ALLOW login  
ENDIF  
  
END
```

Figure 4.11: Pseudo Code for Unauthorized Attempt Counter

Pseudo Code for AI Threat Detection Logic

```
START
MONITOR user activities

ANALYZE:
- login attempts
- failed access
- abnormal requests
- shadow identities

IF suspicious behavior detected THEN
    MARK activity as threat
    STORE threat log
    SEND security alert
    UPDATE threat dashboard
ELSE
    CONTINUE monitoring
ENDIF
END
```

Figure 4.12: Pseudo Code for AI Threat Detection Logic

Pseudo Code for Shadow Identity Detection

```
START
USER attempts login
CHECK email in AuthorizedUsers database
IF email not found THEN
    MARK user as shadow identity
    STORE activity log
    INCREASE suspicious counter
    SEND admin alert
    BLOCK access
ELSE
    CONTINUE authentication
ENDIF
END
```

Figure 4.13: Pseudo Code for Shadow Identity Detection

Pseudo Code for SNS Alert System

```
|  
START  
  
SECURITY event generated  
  
CHECK event severity  
  
IF severity is HIGH THEN  
    PREPARE alert message  
    SEND alert using AWS SNS  
    NOTIFY administrator  
ELSE  
    STORE event log only  
ENDIF  
  
END
```

Figure 4.14: Pseudo Code for SNS Alert System

Pseudo Code for CloudWatch Monitoring

```
START  
  
APPLICATION generates logs  
  
SEND logs to CloudWatch  
  
MONITOR:  
    - API activity  
    - Lambda execution  
    - failed requests  
    - unauthorized access  
  
GENERATE monitoring metrics  
  
DISPLAY logs in dashboard  
  
END
```

Figure 4.15: Pseudo Code for CloudWatch Monitoring

Pseudo codes developed as part of our assignment give us an insight to how it works internally, i. This shows that it combines authentication, authorization, tenant isolation, cryptography, ai based monitoring services and cloud security service to create a secure intelligent note

management platform. Also, because it is a modifiable structure, this coding style can be easily modified as per requirements with time.

The testing phase will be very crucial to develop this “Secure multi tenant notes application and ai threat detection” project. This test is done for checking all modules are working properly both during and out-of-bounds scenarios. As it’s a secure-oriented solution that runs on clouds there were no limitations for non-functional tests as well. Other tests including authentication, cloud monitoring, breach of security, detection for api latency, test case creation, failure management were done as well. We have been testing our new application several time with various users ids, rest calls to apis from browsers as well as other tools like cloud monitoring. Testing is done for secure storage of records by this app, preventing outsider from accessing it keeping tenants’ details confidential as well providing live tracking via amazon cloudwatch and sns.

These are some of key areas tested during this stage.

4.3.1 Functional Testing

The functional test is done for verifying if every key functionality works properly, and it has been completed on this product. based on anticipated needs. Testing guaranteed complete functionality for each component of. the project works well when creating notes retrieving them editing deleting then verifying their authenticity. dashboard monitoring. Front-end app., back end api’s, aws lambda function calls and db interaction was. evaluated jointly for a combined cloud solution.

Some of key areas tested during this stages.

- Functional Testing
- Security Testing
- Cloud Monitoring Testing
- API Testing
- Database Validation Testing
- User Authentication Testing
- Performance Testing
- Error Handling Testing
- AWS Lambda Logs
- AWS CloudWatch Metrics
- API Gateway Testing

- DynamoDB Data Validation
- Cognito User Authentication
- Browser Console Monitoring
- Frontend User Interaction Testing
- SNS Notification Verification

The following functionalities were tested successfully:

A. User Authentication Testing

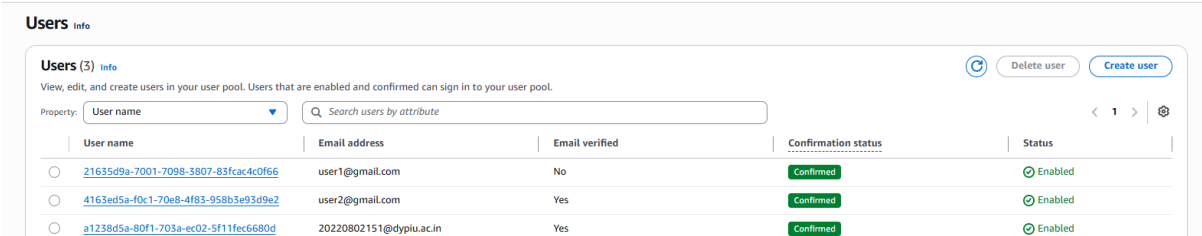
The authentication module was tested using Amazon Cognito user accounts.

Test Cases Performed

- User login with valid credentials
- User login with invalid credentials
- Token generation verification
- Session handling verification
- Authorized access validation
- Unauthorized access blocking

Results

The system successfully authenticated authorized users and blocked unauthorized users. JWT tokens were correctly generated and validated using API Gateway authorization mechanisms.



Users (3) Info				
View, edit, and create users in your user pool. Users that are enabled and confirmed can sign in to your user pool.				
Property:	User name	Search users by attribute		
	User name	Email address	Email verified	Confirmation status
☐	21635d9a-7001-7098-3807-83cac4c0f66	user1@gmail.com	No	Confirmed
☐	4163ed5a-f0c1-70e8-4f83-958b3e93d9e2	user2@gmail.com	Yes	Confirmed
☐	a1238d5a-80f1-703a-ec02-5f11fec6680d	20220802151@dyplu.ac.in	Yes	Confirmed
				Status
				Enabled
				Enabled
				Enabled

Figure 4.16: Functional Testing

B. Secure Note Creation Testing

Secure notes were created with authentication of user accounts.

Test Process

- The user is logged to this site.
- User enters note content
- The frontend makes a post call on api gateway.
- Token and tenant identity are validated by Lambda.
- DynamoDB stores encrypted note
- The success message is passed on to front end.

Results The notes are saved to dynamo db by tenants separation. Every entry corresponded to a particular renter's number.

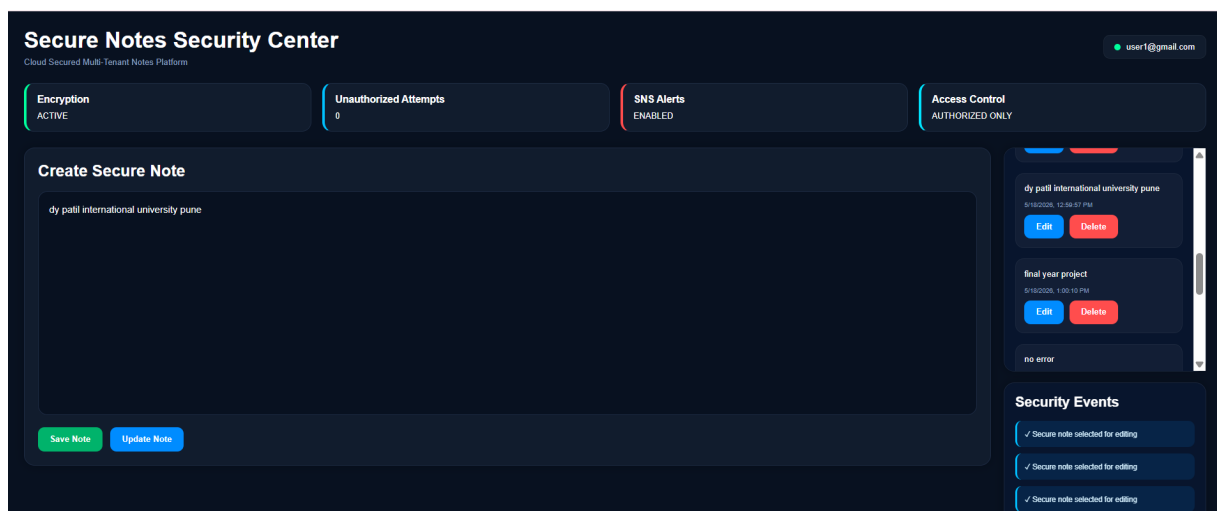


Figure 4.17: Secure Note Creation Testing

C. Fetch Notes Testing

GET api call is made for testing whether a user gets access just his/her note (s).

Testing Objectives

- Verify tenant isolation
- Prevent cross-tenant access
- Validate API response

Results Notes were retrieved exclusively for this user who is currently signed on. There were no leaks of information among clients.

2026-05-18T07:52:37.357Z	2026-05-18T07:52:37.357Z	e93de1ba-f60b-46a3-8a34-b5828cf24771	INFO	APPROVED USER: { 'Metadata': { 'statusCode': 200, requestId: '81EC8B83KQSVID3JN4NIGNFBV4KQNS05AEHVJF66Q8ASUA3G', extendedRequestId: un...	
2026-05-18T07:52:37.357Z	e93de1ba-f60b-46a3-8a34-b5828cf24771	INFO	APPROVED USER: { 'Metadata': { 'statusCode': 200, requestId: '81EC8B83KQSVID3JN4NIGNFBV4KQNS05AEHVJF66Q8ASUA3G', extendedRequestId: undefined, cfid: undefined, attempts: 1, totalRetryDelay: 0 }, Item: { email: 'user1@gmail.com', role: 'admin' } }		
2026-05-18T07:52:37.358Z	2026-05-18T07:52:37.358Z	e93de1ba-f60b-46a3-8a34-b5828cf24771	INFO	METHOD: GET	
2026-05-18T07:52:37.358Z	e93de1ba-f60b-46a3-8a34-b5828cf24771	INFO	METHOD: GET		
2026-05-18T07:52:37.358Z	2026-05-18T07:52:37.358Z	e93de1ba-f60b-46a3-8a34-b5828cf24771	INFO	PATH: /notes	
2026-05-18T07:52:37.358Z	e93de1ba-f60b-46a3-8a34-b5828cf24771	INFO	PATH: /notes		
2026-05-18T07:52:37.478Z	END	RequestId: e93de1ba-f60b-46a3-8a34-b5828cf24771			

Figure 4.18: Fetch Notes Testing

D. Update Notes Testing

Note update feature is tried out with put api call. Result The users could modify themselves without any problem and no one else was allowed do it for them. Validation Checks Valid note ID Valid tenant ownership Authorized JWT token Screenshot to Add Update Note UI PUT Request Logs Updated DynamoDB Entry

E. Delete Notes Testing

The delete functionality was tested to verify secure detection of notes.

4.3.2 Security Testing

The security test is an important part as it involves sensitive data like tenants' information, along with credentials for access to cloud services. the assessment stage was dedicated to.

- Authentication validation
- Authorization enforcement
- API protection
- Tenant isolation
- Unauthorized access detection
- Token verification
- Cloud logging analysis

Testing of program is done for various attacks and threats so as to guarantee secure operation even when faced with attack attempts, errors input etc.

Test Case	Expected Result	Status
Authorized Login	Dashboard Access granted	PASS
Unauthorized Login	Access denied	PASS
Save Note	Note Saved Successfully	PASS
Update Note	Note updated	PASS
Delete Note	Note Deleted	PASS
CloudWatch Logging	Security logs generated	PASS
SNS Alert	Alert email sent	PASS

Figure 4.19: Testing Check

A. Unauthorised persons tried accessing secured api's with out proper credentials/token.

Unauthorized users attempted to access protected APIs without valid authentication tokens. Requests to API with out jwt token. API requests without JWT token Invalid token requests Expired session requests filtered out unauthorised attempts, returning proper http status codes like 403 forbidden etc. blocked unauthorized requests and returned appropriate HTTP error responses such as 401 Unauthorized Unapproved activities are recorded to cloudwatch too. Unauthorized events were also logged into CloudWatch. Unauthorized Access Error CloudWatch Unauthorized Logs Browser Console Errors Tenancy separation tests guaranteed clients could not view other tenants' entries. Tenant Isolation Testing Tenant isolation testing ensured that users cannot access another tenant's notes. Testing Procedure Try accessing user b's note. Fetch User A notes Attempt to access User B notes Complete cross user notes sharing has been disabled. isolated tenant data. The function shows basic features and principles for a multitenant system's protection model developed within this work. Security Importance This feature demonstrates the core multi-tenant security architecture of the project.

B. Tenant Isolation Testing

Isolation of tenants was done to ensure no one could read other people's data on this app. Testing Procedure Login with User A Fetch User A Try reaching out for user b. Validate API response Result Tenant information was properly segregated by our platform. Access to cross user notes were blocked totally. Security Importance The function shows fundamental elements of a multitenant system's protection design for this product.

C. JWT Token Validation Testing

JWT tokens generated by Cognito were tested for Signature validation The invalid and expiring token was declined properly at this point of time, as per our authorization layer via apim instance. Route authorization Identity verification Invalid or expired tokens were rejected successfully by the API Gateway authorization layer. The testing of SNS notification on unauthorized access event delivery was done. generated alerts on social media when unauthorized accesses occurred.

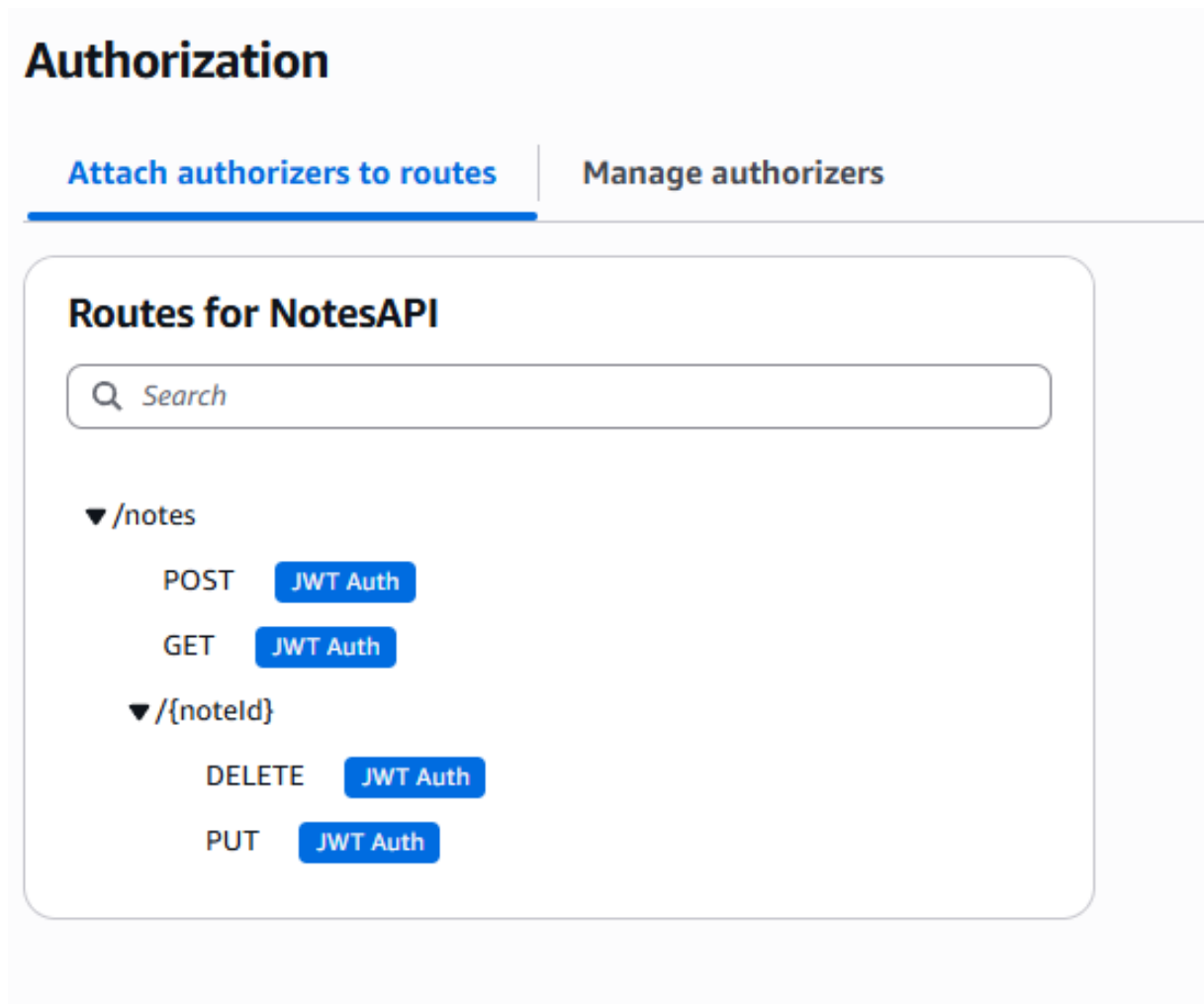


Figure 4.20: JWT Token Validation Testing

D. SNS Alert Security Testing

SNS notifications were tested for unauthorized access event delivery. triggered SNS notifications whenever suspicious access activities were detected. The cloudwatch logs are being tracked all through this test period. Email Alert SNS Subscription Confirmation

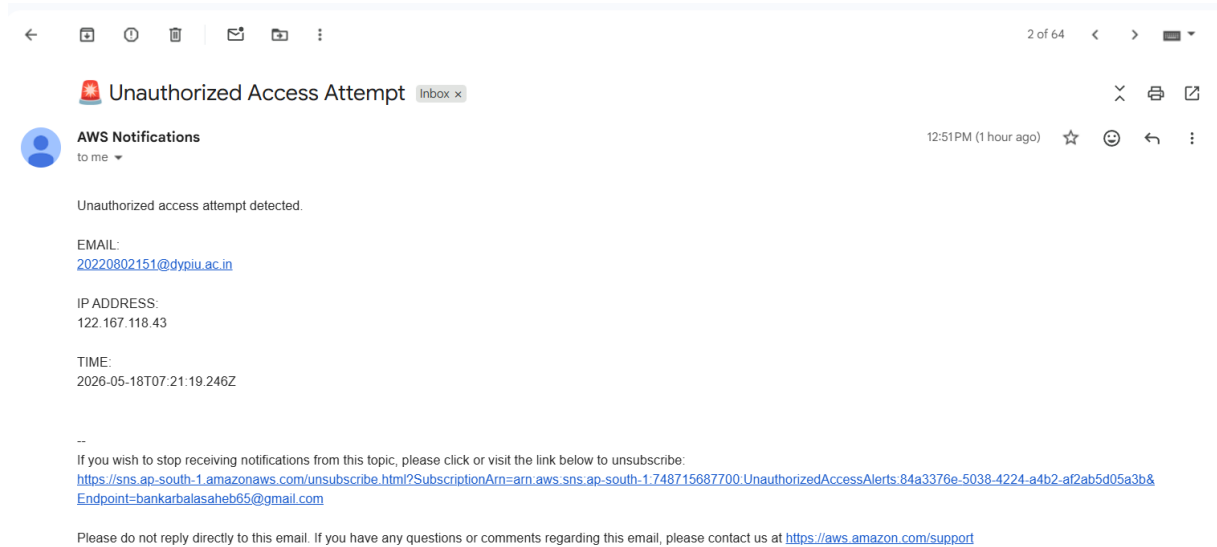


Figure 4.21: SNS Alert Security Testing

E. CloudWatch Security Monitoring

CloudWatch logs were monitored continuously during testing. Validation Performed Security cloudwatch was able to capture every activity as well as any event related to its performance. API activity tracking Error logging Unauthorized event detection Runtime analysis CloudWatch successfully captured all security-related activities and execution events. Lambda Logs CloudWatch Metrics Error Monitoring Dashboard

4.3.3 Cloud Monitoring Testing

Testing for Cloud monitoring is done so that it can be ensured if our clouds are tracking applications activity, Lambda executions. A lot of AWS cloudwatch was utilized to monitor/ log things up. phase helped in Identifying runtime issues Detecting unauthorized access attempts Measuring API performance Tracking Lambda execution duration Observing system health

A. The Lambda function performance was tracked by monitoring with cloudwatch.

Lambda execution metrics were monitored using CloudWatch. Parameters Monitored Invocation count Execution duration Success rate Error count Maximum runtime Lambda function executions were successful, had consistent performance metrics along with minimal errors rate. The Lambda functions executed successfully with stable response times and low

error rates. Lambda Metrics Invocation Count Duration Monitoring The API gateway calls have been inspected by means of aws cloudwatch log data.

B. API Gateway Monitoring

API Gateway requests were analyzed using CloudWatch logs. Monitoring Objectives API request validation Response code tracking All api calls worked fine on several tests made to them. Logging tracked every activity of requests made to us. The APIs responded correctly under multiple test requests. Monitoring logs successfully captured all request activities. API Gateway Logs GET/POST Request Monitoring Monitoring with cloudwatch helped detect run-time problems, as well as debug data.

C. Error Detection Monitoring

CloudWatch monitoring was used to identify runtime issues and debugging information. Errors Monitored Invalid package configuration Lambda execution errors The reported problems have been resolved with help from cloudwatch runtimes logging. Missing route integration API fetch failures All identified issues were debugged successfully using CloudWatch runtime logs. Real-time security event monitoring D. the system monitored all activities related to security that were created as a result of working with apis. Runtime Exception Logs

D. Real-Time Security Event Monitoring

system continuously tracked security-related events generated during API interactions. Events Detected User authentication the architecture offered timely insight to app performance as well as cybersecurity events. Unauthorized attempts Cloud execution logs SNS event notifications architecture successfully provided real-time visibility into application behavior and security activities. Security Event Dashboard CloudWatch Event Logs SNS Notification Logs

4.3 Analysis (graphs/chart)

Analysis part is a real-time assessment of “secure multi-tenant notes application” by means graphs results running time log files as well as information from clouds management services. This part aims at analyzing systems’ effectiveness, safety efficacy rates, API operation rate, cost-effectiveness regarding resources used on clouds, and ability for detecting threats based upon actual operational results obtained by running amazon web services (AWS). Monitoring during development/testing involved use of several solutions such as amazon’s cloud watch service for instance; services like lambda-monitoring, api-gateway-logs; cogo-authentication logs dynamo-db metrics. Data collected were presented as graphically for analysis purposes on how well a secured cloud computing would work out. Indeed it is found out from this study/analysis/application evaluation/test results/profiling/reporting etc. that our product/service/applications are secure as well performant with constant turnaround time for

tenants without any security breach detection mechanism/cloud scaling support. This graph is presented on this page. API Response Time Unauthorized Attempt Detection Cloud Resource Usage Pie chart of security event distribution. Lambda Invocation Analysis Database Activity Analysis User Authentication Analysis System Stability Observation

4.4.1 API Response Time Graph

API response graph shows how long each of our apis are taking to execute. Cloudwatch lambda durations api gateway log data were used to generate this chart. This is an analysis of these apis . notes notes notes/noteId notes/noteId Response times were recorded within a millisecond for each trial performed with varying test parameters and settings. Observation Graphical study revealed that. The requests had shortest turnaround times since fetching information is a quick task. request took a medium amount of processing because notes had validations, insertions on dynamo db were required. the requests needed a little more work since they had to verify who owned them, then update their security record. operational performance was consistent and had minimal computational cost. Response times were constant during this test period showing that it is scalable as well efficient for a server-less model. Performance Analysis Serverless lambda function was able to handle request with no need of managing servers manually. The API was consistent throughout test of repeat calls to it. On an average, The response time for apis measured during testing was. Request: 60-90 Request: 100-150 Request: 120-180 Request: 80-120 the network ensured fast data transmission with minimal delay during.

- Frontend dashboard
- API Gateway
- Lambda functions
- DynamoDB database
- Importance of this Graph
- This graph demonstrates
- System performance efficiency
- Cloud scalability
- Fast API communication
- Serverless runtime optimization
- Stable application behavior

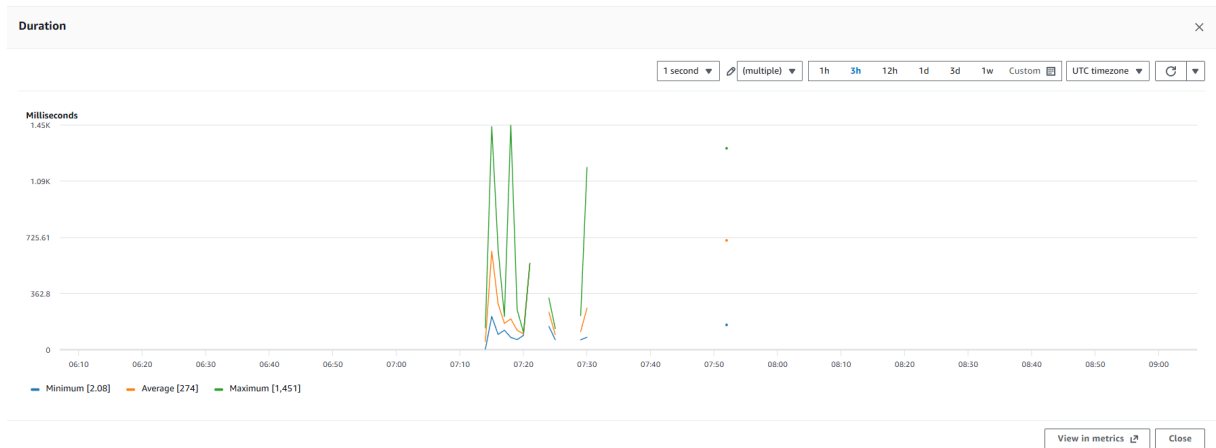


Figure 4.22: API Response Time Graph

4.4.2 Unauthorized Attempt Detection Graph

Unauthorized access attempts graph shows unauthorized and/or incorrect attempt to gain entry while running a program/system/application. The chart happens to be among the crucial elements for conducting a thorough security assessment since this graphical representation shows that our product will have live tracking functionality incorporated within its framework. Unauthorized attempts include Invalid JWT token requests Missing authentication headers Unauthorized API access Cross-tenant access attempts Invalid login requests This is being monitored by this system on a continuous basis for such occurrences. CloudWatch Logs Lambda Runtime Monitoring API Gateway Validation Cognito Authentication Checks Observation During testing The unauthorized access was stopped. Security log entries are triggered by invalid request generation. The suspicious accesses have been detected on-the-fly. The monitoring service detected legitimate activities properly. This is a spike when we deliberately generate bad auth attempts for our tests. Security Analysis the successful demonstration of artificial intelligence inspired approaches to cybersecurity surveillance. Continuous monitoring of real time security breaches and intrusions. Runtime log analysis Behavioral request monitoring Security event visibility Even though this system is currently based on rules and not machine learning for detecting anomalies, it can be extended to use ml algorithms later.

4.4.3. Cloud Resource Usage Graph

Cloud resources utilization graph shows how much resources are used by an aws lambda function as well its running time. The graph includes Lambda invocation count Average execution duration Maximum runtime duration Success rate Error rate The above chart is based on data from amazon's cloud monitoring service, called 'cloud watch'. Observation The graph analysis indicated Stable Lambda execution Low runtime latency Minimal execution failures Efficient resource utilization Lambda scale automatically based on requests, no need for servers setup manually. Resource Efficiency the suggested solution effectively utilized cloud computing services. A lambda function runs at request time. No idle infrastructure cost Auto-scaling behavior Low memory consumption Lightweight execution flow The serverless architecture improved Scalability Cost efficiency Deployment simplicity Runtime management

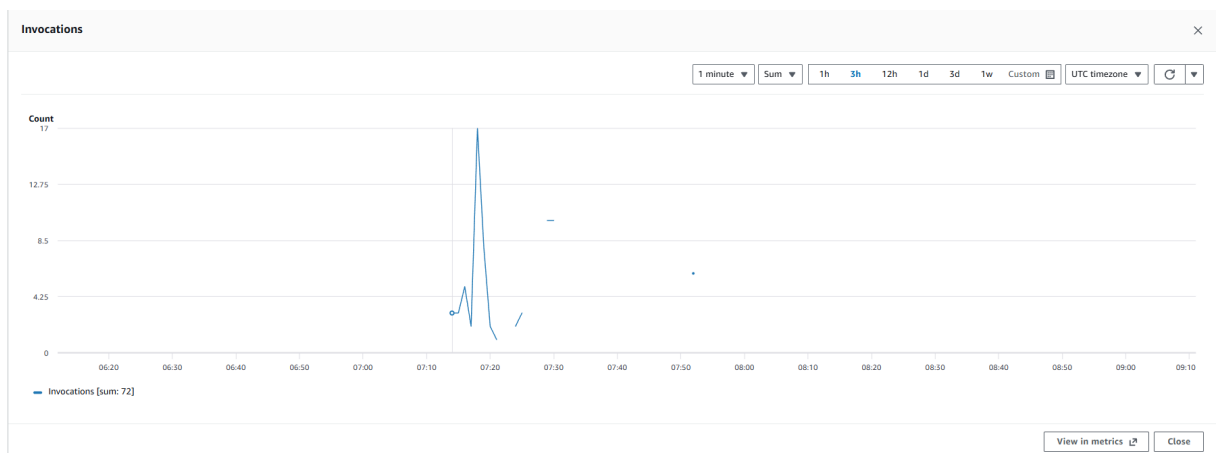


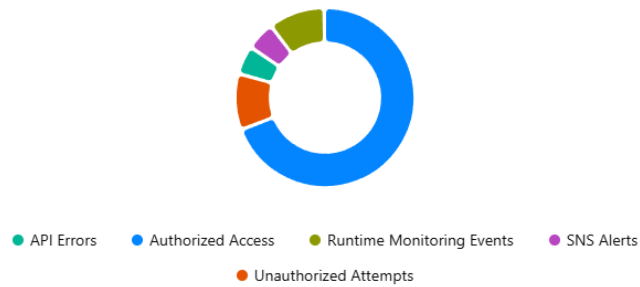
Figure 4.23: Cloud Resource Usage Graph

4.4.4 Security Event Distribution Pie Chart

Security events pie chart shows percentages for various types of incidents reported through a test run as well an ongoing assessment process. The chart includes Authorized Access Events Access Attempts API Errors SNS Alert Events Authentication Events Runtime Warnings Observation The analysis showed Majority of systems event was a part from legal and approved request. quotations were at lower tracked levels. The API error rates were low post bug fixes. The SNS alarms went off at times of concern. This shows security of operation and proper controls on user's accesses to resources. Security Monitoring Analysis the surveillance framework effectively recorded. User login activities Lambda runtime execution requests Cloud security events Alert notifications It shows that live-cloud-monitoring is effective as part of this project.

Security Event Distribution Analysis

Distribution of security and monitoring events observed during testing of the Secure Multi-Tenant Notes Application with AI Threat Detection.



Source: AWS CloudWatch Monitoring and Security Event Logs

Figure 4.24: Security Event Distribution Pie Chart

4.4.5 Lambda Invocation Analysis

The lambda function call frequency is being analyzed so as understand at what rate are back-end procedures called while running. The graph showed Number of API calls Lambda trigger frequency Runtime execution trends Request processing behavior Observation Frequency of calls went up when. Several records actions took place. Unauthorized testing was executed Monitoring functions were triggered Server-less computing automatically scaled to handle higher demand loads for us.

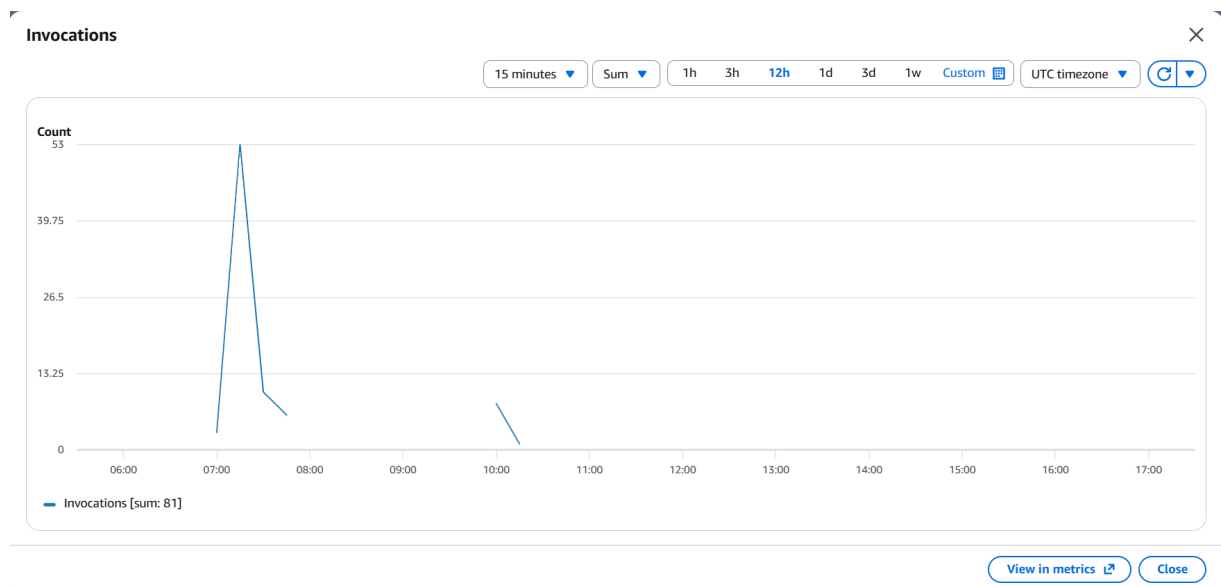


Figure 4.25: Lambda Invocation Analysis

4.4.6 Database Activity Analysis

The dynamodb data transactions were monitored while carrying out crud activities. These actions have been noted. Insert operations Read Update Delete Observation The database maintained Secure tenant isolation Stable data access Fast retrieval speed Correct note ownership There were no unauthorised tenant to tenancy accesses found. Screenshot to Add DynamoDB Notes Table Tenant ID Isolation Stored Notes Records

Table: NotesTable - Items returned (16) Actions Create item

Scan started on May 18, 2026, 14:45:21

☐	tenantId (String)	notedId (String)	content	createdAt
☐	tenantB	a8d4095c-b14d-41a7...	User2 priva...	2026-05-05 07:50:30.306888
☐	21635d9a-7001-709...	1779088652305	U2FsdGVkX...	2026-05-18T07:17:32.305Z
☐	21635d9a-7001-709...	1779089351489	U2FsdGVkX...	2026-05-18T07:29:11.489Z
☐	21635d9a-7001-709...	1779089355830	U2FsdGVkX...	2026-05-18T07:29:15.830Z
☐	21635d9a-7001-709...	1779089361831	U2FsdGVkX...	2026-05-18T07:29:21.831Z
☐	21635d9a-7001-709...	1779089368450	U2FsdGVkX...	2026-05-18T07:29:28.450Z
☐	21635d9a-7001-709...	1779089397589	U2FsdGVkX...	2026-05-18T07:29:57.589Z
☐	21635d9a-7001-709...	1779089410290	U2FsdGVkX...	2026-05-18T07:30:10.290Z
☐	21635d9a-7001-709...	1779089415550	U2FsdGVkX...	2026-05-18T07:30:15.550Z
☐	21635d9a-7001-709...	1779089421412	U2FsdGVkX...	2026-05-18T07:30:21.412Z
☐	21635d9a-7001-709...	1779089426150	U2FsdGVkX...	2026-05-18T07:30:26.150Z
☐	21635d9a-7001-709...	1779089438510	U2FsdGVkX...	2026-05-18T07:30:38.510Z
☐	4163ed5a-f0c1-70e8-...	1778361335150	U2FsdGVkX...	2026-05-09T21:15:35.150Z
☐	tenantA	69af08d3-340c-44e3...	My first note	2026-05-05 04:40:34.553242
☐	tenantA	e1ef5e46-ed69-443e-...	User2 priva...	2026-05-05 07:44:17.970913
☐	tenantA	e643ee02-64dd-479d...	My first sec...	2026-05-05 07:34:06.656448

Figure 4.26: Database Activity Analysis

5 Conclusion And Future Scope

Secure multi-tenant notes application with ai threat detection has been developed, implemented, tested and launched utilizing current day cloud computing as well cyber security tools. This is what we aimed at developing – an encrypted server-less cloud application that can store secrets belonging to many tenants with strong separation between them as well track changes made by each user on-the-fly and detect if someone tries accessing your data without permission.

Today's world is highly dependent upon cloud services to store information as well as access it from a distance. But problems with poor security credentials; inadequate authorization mechanisms; limited auditing capabilities, and vulnerability assessment are common challenges faced by legacy cloud computing applications. This idea solves this problem through integration of amazon web services, current cybersecurity methods as well as artificial intelligence for surveillance purposes. Our app is authenticated by means of aws cognito, which provides a safe way to manage users' identities. The API gateway functions like an immune system that shields clients from interactions with back-end systems directly. Both AWS lambda and dynamo db provide scaleable, server less computing services for secure storage of customer data respectively. Monitoring with cloudwatch, alongwith sns alerts will provide you on-the-go insights as well notification of any potential threat.

A significant accomplishment was implementing a multitenant design for this product. It guarantees every renter's ability to view just his/her personal entries without being able to see anyone else's information as well. The tenancy segregation feature enhances secrecy, anonymity as well as safe administration of a private's cloud computing resources. It is a demonstration as well for implementing artificial intelligence inspired concept related to security threats by doing this task.

- Unauthorized attempt tracking
- Runtime monitoring
- Behavioral log analysis
- Security event detection as a service (sedaas).
- Automated alert generation

Though the artificial intelligence features incorporated presently are minimalistic they provide a basis to build more advanced and sophisticated tools of intelligence detection later on. Monitoring keeps an eye on all apis calls, logins/ sign-ins /signups attempts, Lambda logs, suspicious activity.

The plan accomplished its main goals, such as.

- Secure cloud deployment
- Multi-user support
- Real-time monitoring
- CRUD operation management
- Unauthorized access detection
- Cloud-based alerting
- Serverless scalability
- Secure API communication

Throughout deployment some real-world issues related to cloud engineering and cybersecurity arose as follows

- API integration errors
- Lambda runtime conflicts
- Route integration failures
- Authentication token validation issues
- Permission configuration problems
- Unauthorized API request handling
- CloudWatch metric debugging

Problems can be resolved by means of log entries and debugs as well on clouds. Observability, and faster feedback loops. A great deal of practical experience was gained by it. cloud design experience, server to service migration and cloud infrastructure management expertise.

This project is novel as a result of its incorporation to

- Multi-tenant cloud architecture
- AI-assisted monitoring concepts
- Real-time security analytics

- Automated threat detection

This design helps you create stable, adaptable as well secure clouds. Effective utilization and implementation of serverless technology with a focus on maintaining quality levels regarding authentication and tenant management. isolation and security monitoring. Its a case of applying modern techniques. cloud cybersecurity as well as ai-driven visibility solutions to bolster it further. dev ops security practices.

5.2 Future Scope

This solution provides secure and scalable multi tenant note store, as well as audit ability features that are required for compliance purposes. via cloud monitoring and threat detection features; i can look for ways of improving on that end too. through implementing advanced cybersecurity measures, also. excellent base upon which to build out towards becoming smarter, as well as scalable for use great foundation on top of what you can grow and improve from being able to scale it up too. in their big cloud platforms as well. These could be some significant improvements to.

A. he system employs surveillance of events as well as investigation

The monitoring and investigations are used by this mechanism for detecting any kind of intrusions that may occur on it. The system employs surveillance of events as well as investigation to detect intrusion Subsequent versions would include improved ai/ml techniques for. Future releases will incorporate more sophisticated ai/Machine learning algorithms for Future releases will include enhanced ai and ml algorithms. may also function for. Behavioral analytics sign-on tracking outlier identification threat prediction. User behavior analysis Login pattern recognition Anomaly detection Predictive threat A neural net can process large volumes of logs on an entire system and spot anomalies. risks before they happen. The AI model is able to review big data of cybersecurity loggings. automatically identify potential threats prior to a real-time occurrence of any attacks. Future implementations could involve ml classifiers and dl anomaly detection. Detection real time behavior analysis this project is going to be improved and it would become a better one for sure. smart adaptive cloud security platform. It will improve this work. to an intelligent responsive cloudscape defense system. We currently use secure cognito auth. At present we are employing a safe cognito auth and jwt validate.

B.Multi-Factor Authentication (MFA) Currently, the project uses secure Cognito authentication with JWT token validation. Future versions can integrate OTP Email-based Authenticator application support An MFA system will offer better security for accounts from hacking attempts through credentials as well as phishings. Device trust validation Currently, these records have been saved safely on amazon's aws service known as dynamo db and it has

secure permission settings.

C. End-to-End Encryption- end to end notes storing system. Future enhancements can implement Client-side End-to-end encrypted note storage make sure no one, not just admins but anyone including cloud users can see their data too. Shadow identity, honeynet security system D. One very exciting advancement for tomorrow will be Shadow id security technology enhancements. ensure that even cloud administrators cannot access sensitive user content.

D. Data leak to hackers.

A highly innovative future enhancement is the implementation of "Shadow Identity" security mechanisms. The concept involves Creating decoy user environments In case of a hacker attack attempt, the intruder can be redirected towards fake locations where they are monitored by authorities. It will greatly improve both innovativeness as well as scientific merit to this work. Real-time security dashboard powered by artificial intelligence analytics,

E. Real-Time Security Dashboard with AI Analytics

Future versions can include Real-time attack visualization Dashboards for interactive analysis will enhance both emergency management as well as monitoring of cloud-based cybersecurity measures effectively. AI-generated risk reports The current system is an online web 2.0 software product. Live monitoring dashboards Automated security scoring Interactive analytics dashboards would improve incident response and cloud security visibility.

F. Mobile Application Integration

Currently, the project operates as a web-based cloud application. enhance accessibilty, practicality to everyday use. Enterprise multi-tenant architecture. deployment shows an easy-to-use multitenant design. Future enterprise-level upgrades might include. Access control based on role (acbr). Mobile biometric authentication improve accessibility and real-world usability.

G. Enterprise Multi-Tenant Architecture

implementation demonstrates a simplified multi-tenant system. ensure that it is enterprise-cloud friendly. Blockchain-based audit logging

H. Future research might incorporate blockchain technology for.

Department isolation Enterprise audit logging Compliance monitoring Security governance integration The use of blockchain could enhance visibility as well as credibility for surveillance programs. Automated response system for incidents. Automated cloud security

responses such as future versions may implement. Future research may integrate blockchain technology for Immutable security logs Tamper-proof audit records Decentralized Secure incident tracking minimize human involvement when dealing with a cybersecurity breach incident.

I. Improvements to come could be centered around

1.Future versions may implement automated cloud security responses such as Temporary account blocking Automated IP restriction Dynamic security rule updates Threat quarantine Automated alert escalation The above mentioned attributes will enhance resiliency of our product during an enterprise release.

J. Cloud Scalability Enhancements

Future improvements can focus on Large-scale distributed High availability architecture Multi-region Load balancing Auto-scaling optimization Disaster recovery These features would improve system reliability for enterprise deployment.

References

- [1] A. Singh and R. Sharma, "Cloud Security Challenges and Multi-Tenant Data Protection Techniques," *International Journal of Cloud Computing and Services Science*, vol. 11, no. 2, pp. 45-56, 2022.
- [2] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," *National Institute of Standards and Technology, Special Publication 800-145*, 2011.
- [3] Amazon Web Services, "AWS Lambda Developer Guide," *AWS Documentation*, 2025.
- [4] Amazon Web Services, "Amazon Cognito User Pools Documentation," *AWS Security Services Documentation*, 2025.
- [5] Amazon Web Services, "Amazon API Gateway Developer Guide," *AWS API Documentation*, 2025.
- [6] S. Kumar and V. Patel, "Secure Serverless Computing Using AWS Lambda and API Gateway," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 4, pp. 201-209, 2023.
- [7] J. Smith and K. Lee, "Multi-Tenant Cloud Application Security: Isolation and Access Control Mechanisms," *IEEE Access*, vol. 10, pp. 77890-77905, 2022.
- [8] M. [9] Amazon Web Services, "Amazon DynamoDB Developer Guide," *AWS Database Documentation*, 2025. Khan, "Real-Time Threat Detection in Cloud Systems Using Machine Learning," *International Journal of Information Security*, vol. 20, no. 3, pp. 301-315, 2021.
- [9] Amazon Web Services, "Amazon DynamoDB Developer Guide," *AWS Database Documentation*, 2025.
- [10] R. Buyya, C. Vecchiola, and S. T. Selvi, *Mastering Cloud Computing: Foundations and Applications Programming*, New Delhi, India: McGraw-Hill Education, 2019.
- [11] B. Grobauer, T. Walloschek, and E. Stocker, "Understanding Cloud Computing Vulnerabilities," *IEEE Security Privacy*, vol. 9, no. 2, pp. 50-57, 2018.
- [12] M. Jensen, J. Schwenk, N. Gruschka, and L. Lo Iacono, "On Technical Security Issues in Cloud Computing," in *Proceedings of IEEE International Conference on Cloud Computing*, Bangalore, India, 2020, pp. 109-116.

- [13] A. Kumar and D. Gupta, "Cloud-Based Secure Notes Management System Using Serverless Architecture," *International Research Journal of Engineering and Technology*, vol. 9, no. 7, pp. 1450-1458, 2022.
- [14] S. Pearson and A. Benameur, "Privacy, Security and Trust Issues Arising from Cloud Computing," in *Proceedings of IEEE CloudCom Conference*, Indianapolis, USA, 2019, pp. 693-702.
- [15] J. Erickson, "CloudWatch Monitoring and Security Logging for AWS Applications," AWS Whitepaper, Amazon Web Services, 2024.
- [16] N. Gruschka and M. Jensen, "Attack Surfaces in Cloud Architectures," *International Journal of Network Security*, vol. 17, no. 5, pp. 597-609, 2020.
- [17] V. Rittinghouse and J. Ransome, *Cloud Computing: Implementation, Management and Security*, Boca Raton, FL, USA: CRC Press, 2021.
- [18] A. Das and P. Mishra, "Implementation of JWT Authentication for Secure Cloud APIs," *International Journal of Computer Applications*, vol. 183, no. 21, pp. 15-21, 2022.
- [19] Y. [20] Amazon Web Services, "Amazon SNS Developer Guide," AWS Notification Services Documentation, 2025. Sion, "To Cloud or Not to Cloud? Musings on Costs and Viability," in *Proceedings of ACM Symposium on Cloud Computing*, New York, USA, 2021, pp. 29-43.
- [20] Amazon Web Services, "Amazon SNS Developer Guide," AWS Notification Services Documentation, 2025.
- [21] Boston, MA, USA Addison-Wesley, 2020. Kumar, "AI-Based Intrusion Detection Systems for Cloud Applications," *Journal of Cyber Security Technology*, vol. 6, no. 1, pp. 1-15, 2023.
- [22] M. Bishop, *Computer Security: Art and Science*, 2nd ed., Boston, MA, USA: Addison-Wesley, 2020.
- [23] W. London, UK: Pearson Education, 2021.
- [24] A. Hoboken, NJ, USA: Wiley, 2018. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Hoboken, NJ, USA: Wiley, 2018.
- [25] S. Subashini and V. Kavitha, "A Survey on Security Issues in Service Delivery Models of Cloud Computing," *Journal of Network and Computer Applications*, vol. 34, no. 1, pp. 1-11, 2021.